Open AR Cloud WebXR Client

Dr. Gábor Sörös

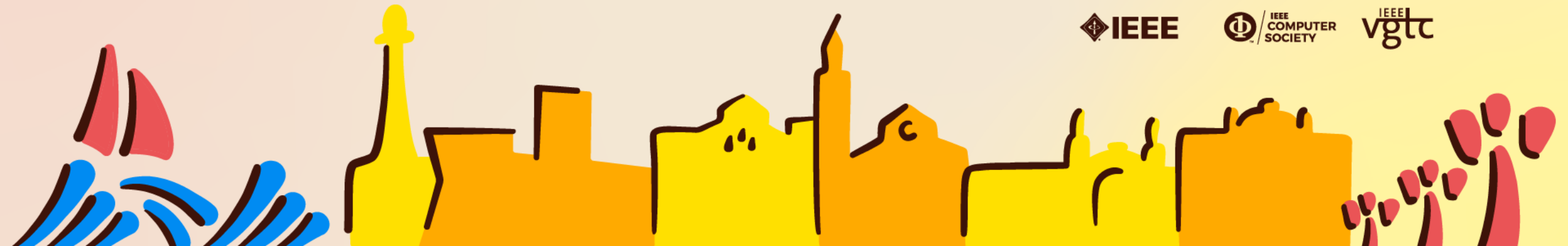
Open AR Cloud / Nokia Bell Labs

ISMAR 2026

October 5 - 9, Bari, Italy

The 25th International Symposium on Mixed and Augmented Reality

IEEE IEEE COMPUTER SOCIETY IEEE vgtc



Vision: AR cloud

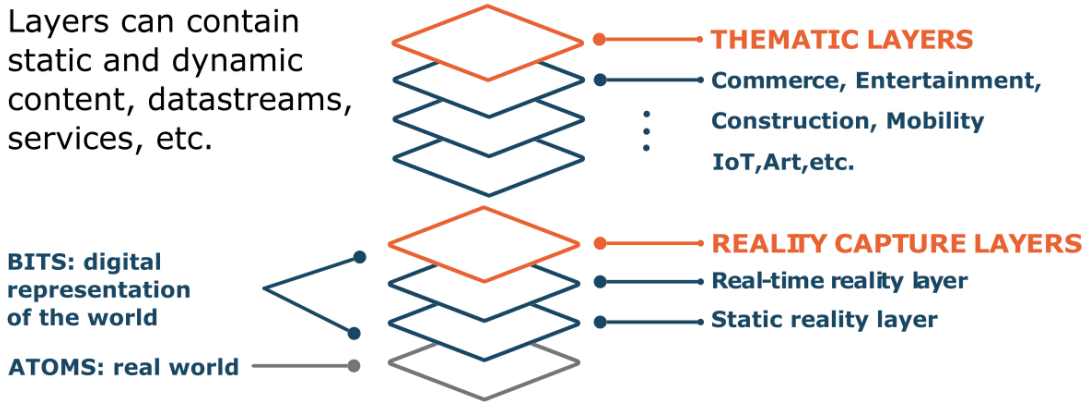
Augment the world with persistent digital objects

- Objects anchored to geo-locations
- Global 6DoF localization of client devices
- Collaborative mapping
- Geospatial query of contents
- Collaborative authoring: anyone can create
- Open: anyone can browse contents
- Real-time and interactive presentation

Open Web → Open Spatial Web?

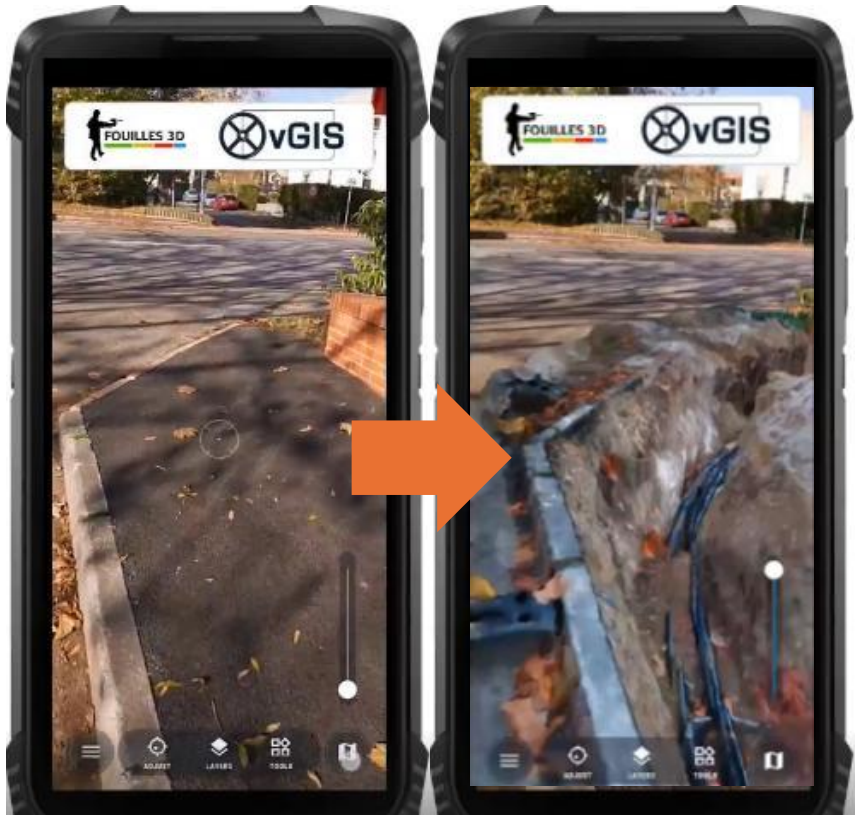


Source: https://www.rypplzz.com/images/westfield-05.jpg



Examples: real-time information & situated visualization

Location-based information on the go



vGIS Fouilles3D: GIS visualization



Immersal: train schedule



Snap: restaurant data in London

Why WebXR?



+ runs in browser, no need to install anything
+ triggering AR experiences becomes very easy, seamless activation
+ universally browsing various types of digital contents
+ (ideally) wide range of devices supported

- but: at experimental stage, very limited browser support
- currently **Android Chrome** full functionality, Edge partial functionality, Safari ??
- starting the camera requires explicit button press, and `camera-access` feature is mandatory



SPARCL = spatial AR cloud client

`https://github.com/OpenArCloud/sparcl`

Digital contents attached to GeoPose

- primitives from textual descriptions
- GLTF/GLB models (texture OK, but no complex shaders)
- icons
- OGC Point of Interest
- iframe: complex scene loaded via URL
- OGC GeoPose streams → Metaverse Standards Forum



Try it yourself!

https://sparcl.orbit-lab.org

Try it!

your own content here!

1. Enable WebXR in Android Chrome

`chrome://flags`

Search for 'WebXR Incubations', → Enabled

2. spARcl is deployed at:

`https://sparcl.orbit-lab.org`

(use HTTPS)

3. Enable Permissions

by clicking on the lock icon in the URL field (the app will also ask for them): Camera, Location

4. Swipe through the intro

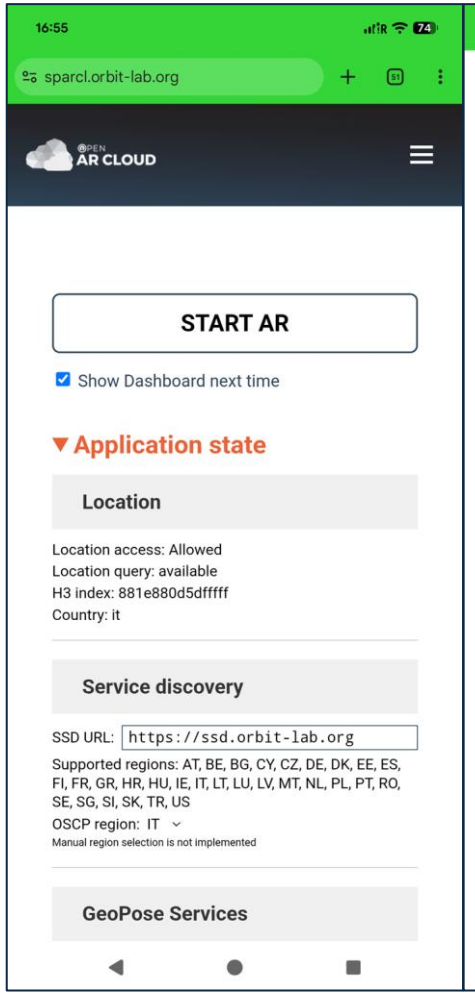
the camera starts automatically. Press back to come to the Dashboard

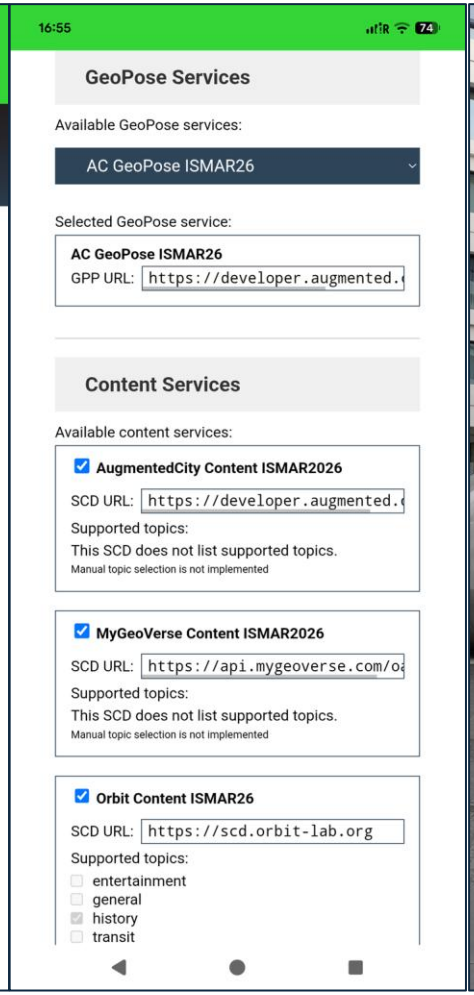
5. Select services

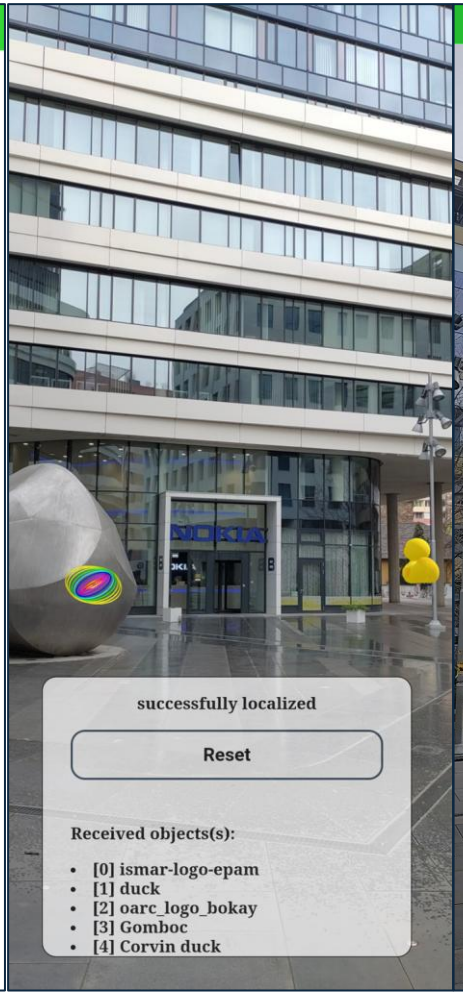
review the gathered information. Select AC Geopose, Orbit content, XRM content, etc.

6. Go immersive

Look around, Localize, Enjoy









Setting up the code 1/2

Prerequisites:
git, npm (Node.js), code editor (ex. Visual Studio Code)

Code and dependencies:

```
cd ~/dev/OpenARCloud
git clone https://github.com/OpenArCloud/sparcl
cd sparcl
npm install
```

Enable local HTTPS server:
HTTPS is a must because we deal with camera and location access

Set Service Discovery root URL in .env file:
`VITE_SSD_ROOT_URL= <...>`

vite.config.js

```
plugins: [
        svelte(),
        glsl(),
        basicSsl(),
        typescript(),
    ],
```

Setting up the code 2/2

Enable WebXR in Android Chrome

`chrome://flags`

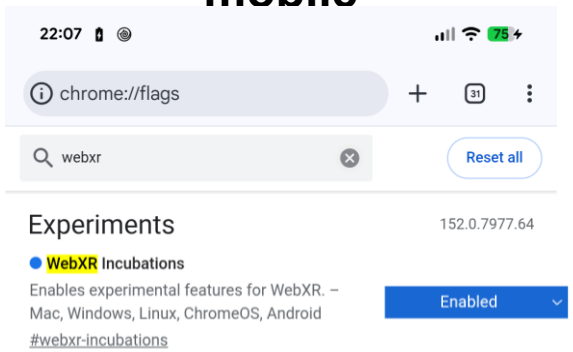
Search for ‘WebXR Incubations’, set to Enabled

Chrome remote debugging

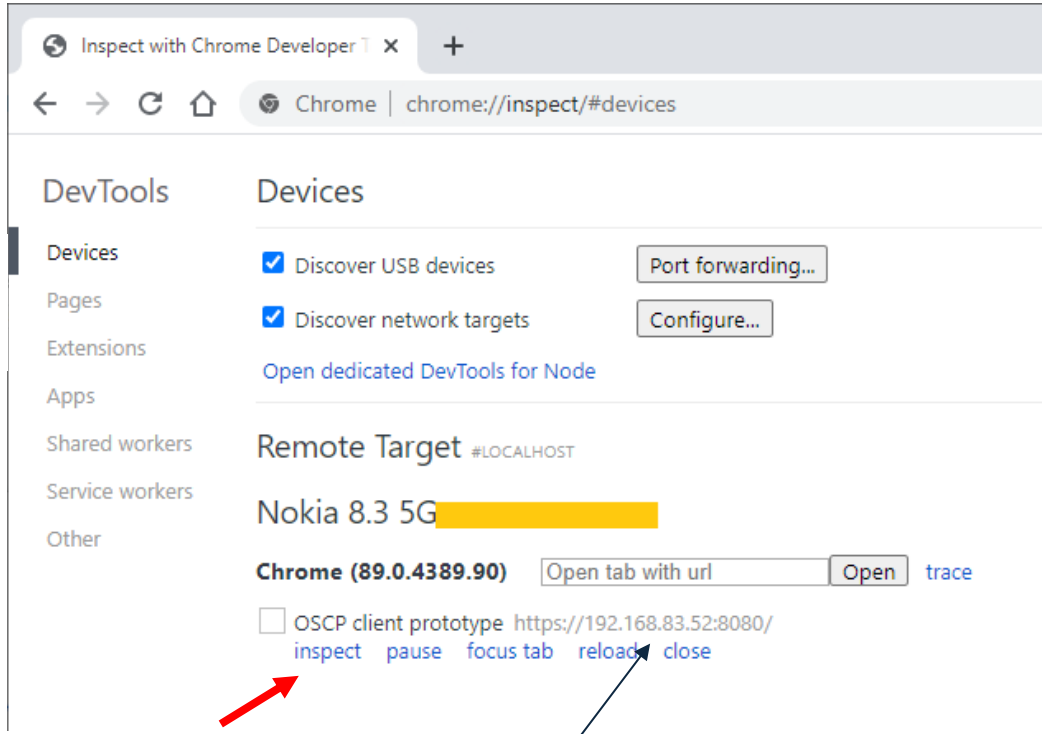
`chrome://inspect/#devices`

Attach phone via USB cable or enable Wireless debugging in the Developer options

mobile



desktop



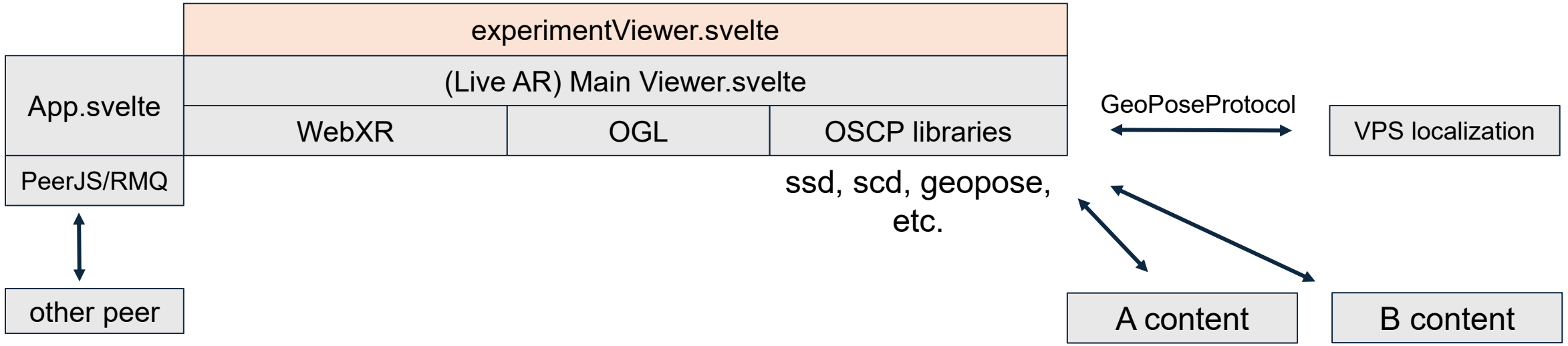
Finally, start the Web server:

```
cd ~/dev/OpenARCloud/sparcl
npm run start
```

note the URL in the output (make sure your phone and server are on the same network)

Application architecture

Svelte: JavaScript frontend framework for building user interfaces with many advantages compared to Vue and React. Svelte code compiles to JavaScript DOM manipulations, avoiding the need to bundle a framework to the client, and allowing for simpler application development syntax.
Vite: fast build tool with nearly immediate reload or changed JavaScript modules without losing state
OGL: lightweight WebGL 3D engine
PeerJS or RabbitMQ: for multiplayer, will be covered later today



Authentication

Authentication via Auth0.com

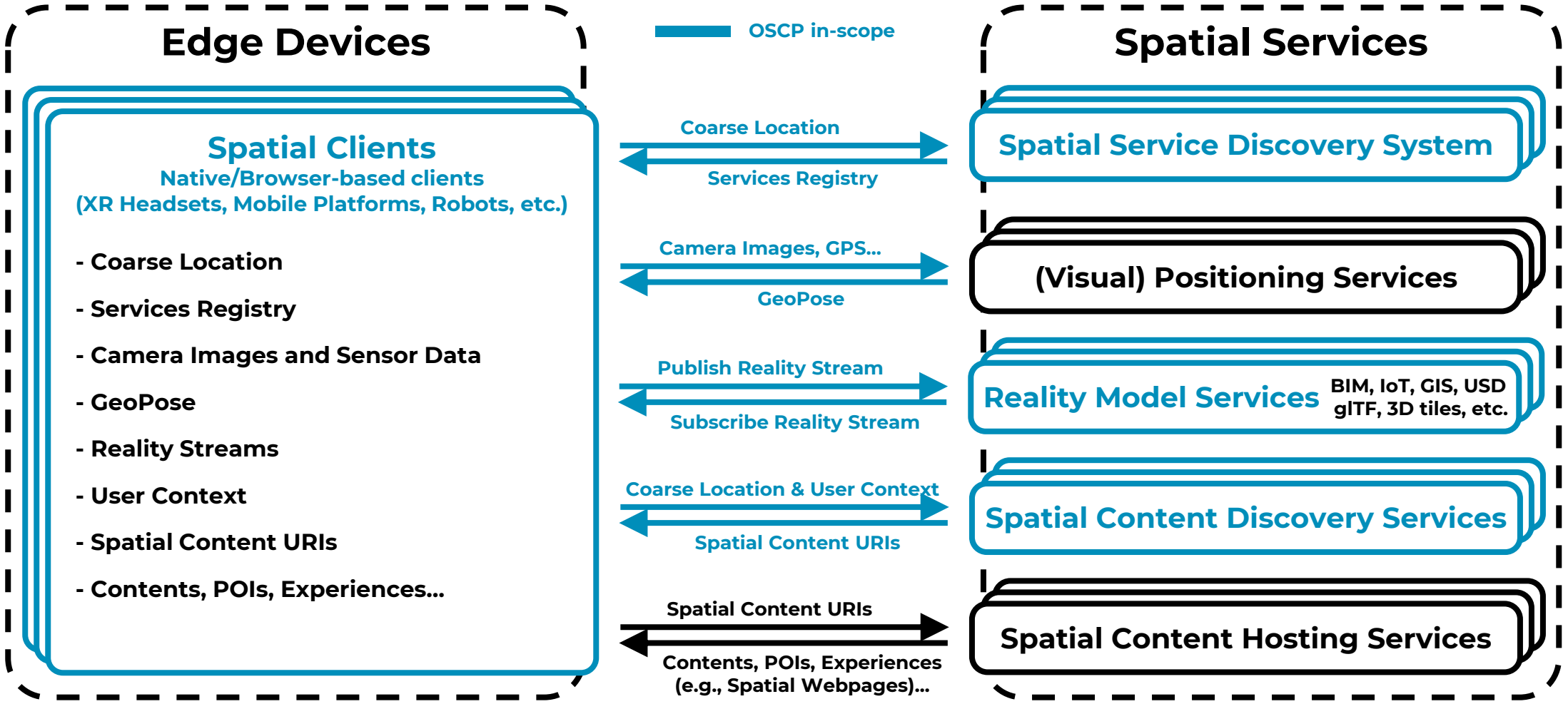
- Required if we want to add contents in the future (but can also be hosted with auth disabled)
- Username automatically extracted

`VITE_NOAUTH=true` in env file disables authentication

We use Auth0.com for wrapping several OAuth2.0 providers. For example Google.

If you want to use Auth0, you can either create your own tenant and app, or ask us for our settings.

OSCP 3.0: SEQUENCE DIAGRAM



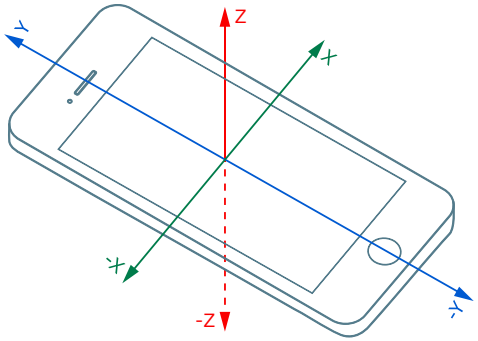
Localization with a supported VPS

You or we need to create the map of your environment first. This means capturing many photos and waiting a few hours.

- OpenVPS: StrayScanner https://youtube.com/shorts/oh5gkDY84FE
- AugmentedCity: AC Scanner https://youtube.com/shorts/U9Pmh6FyzO0
- (Immersal: Immersal Mapper) https://youtube.com/shorts/MqdGf44xn3o

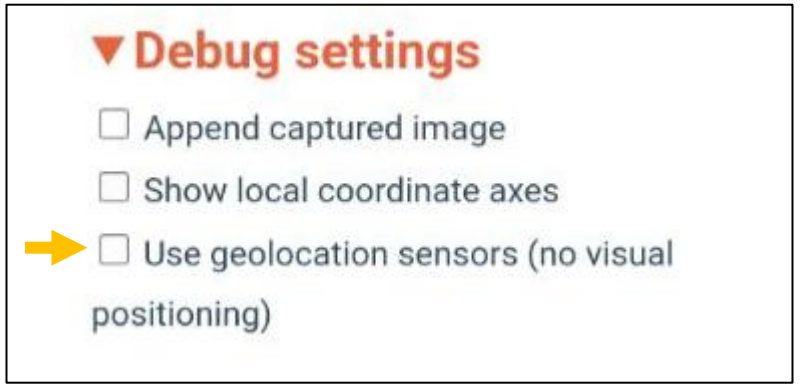
Please contact us for registering the selected VPS provider as a "GeoPose service" at your location to be listed in our spatial service discovery (SSD). Find your intended AR location on Google Maps, right click, and copy the GPS coordinates. We will enable the selected GeoPose service around that location.

Fallback: sensor-based localization



`GeolocationSensor` (fused GPS and RF signal strength)

`AbsoluteOrientationSensor` (fused acc, gyro, magnetometer)



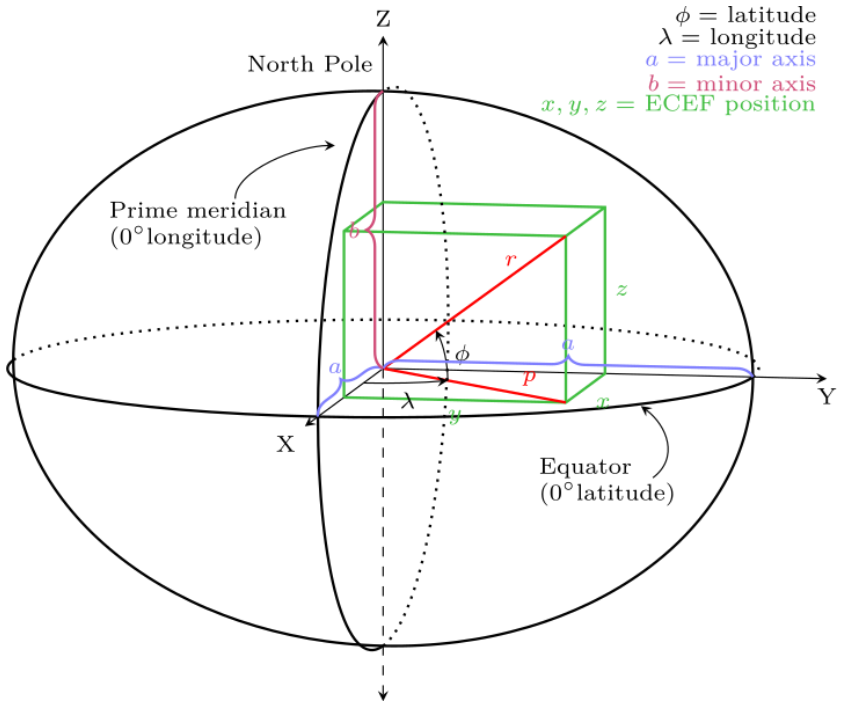
Pros:

- works out of the box
- no mapping needed, no photos taken

Cons:

- inaccurate *(localize repeatedly and observe the deviations)*
- coordinate system may not be consistent across devices *(not suitable for multiplayer)*
- coordinate axes depend on screen orientations *(currently works only in landscape mode)* → *force landscape UI*

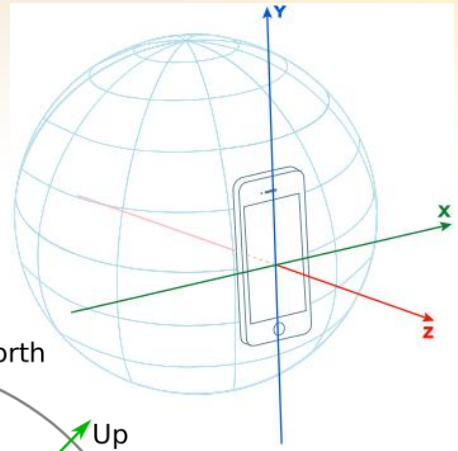
Coordinate systems

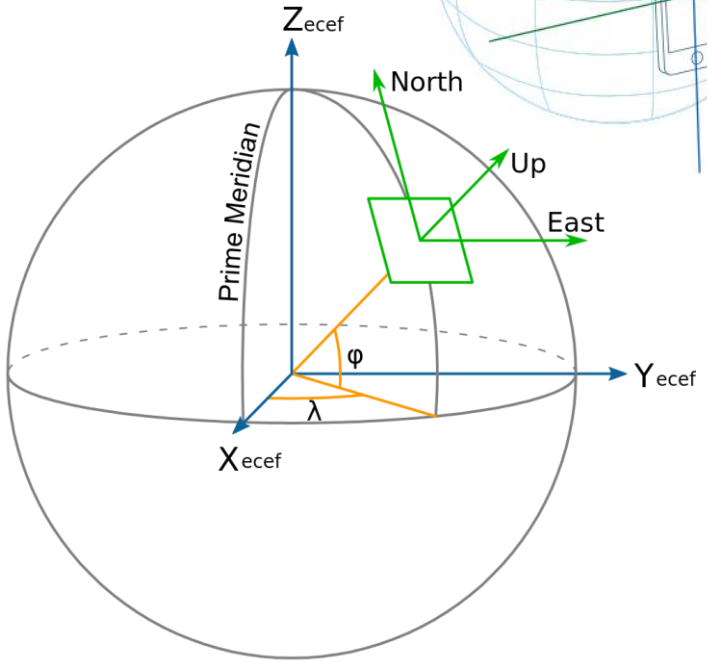


ECEF: Earth-Centered, Earth-fixed

used in GeoPose ecef field

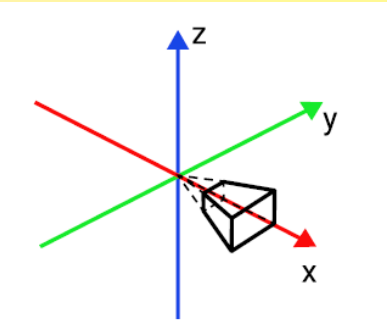
W3C OrientationSensor



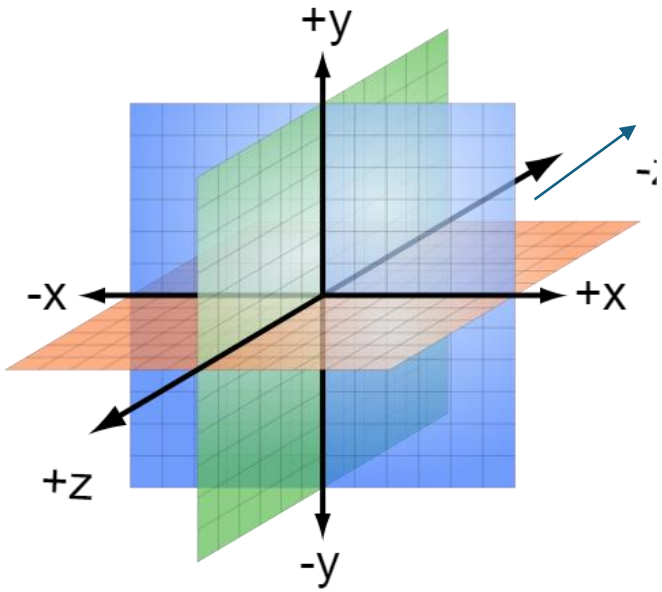


LTP: Local Tangent Plane approximation
ENU: East-North-Up (Z up)

used in GeoPose location field



GeoPose ENU
camera looks towards E/X



WebXR local coordinate system
right handed, Y up

used in GeoPose object orientation field

WebXR camera looks towards -Z

Fallback: fake geolocation

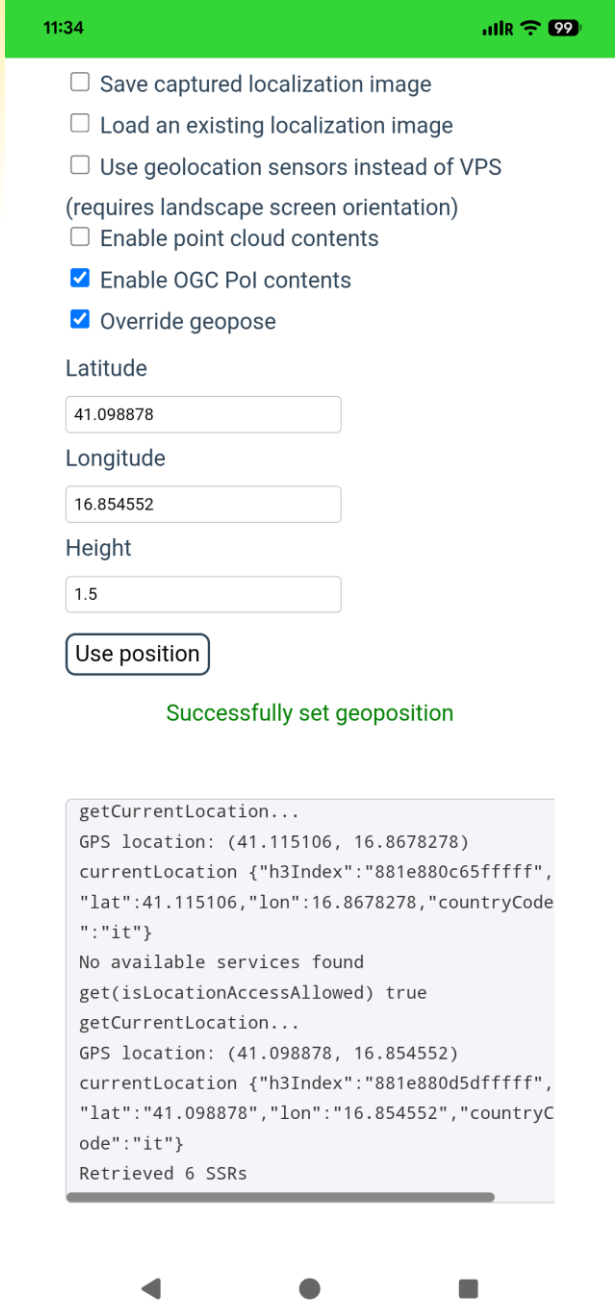
Debug/Override geopose

- triggers service re-query on dashboard
- enter AR
- press Localize, this GeoPose will be returned
- (uncheck the use geolocation sensors)

Coordinates at ISMAR2026:

```
lat 41.098766
lon 16.854622
h 1.5
```





Camera access in WebXR

No access to images from the browser by default.

An extension allows to query the ID of the camera texture used for rendering the background. We can tap on that texture in GPU memory.

sparcl/src/core/cameraCapture.js

```
function createImageFromTexture(gl, texture, width, height)
```

Querying *services* and *contents*

The services for an area need to be registered in a Spatial Service Discovery server

We run a sandbox SSD service at https://ssd.orbit-lab.org

Sparcl queries this server for available services around the user's coarse location

The content entries should be registered in a Spatial Content Discovery (SCD) server that covers the area.

We also run a sandbox SCD service at https://scd.orbit-lab.org

Sparcl queries this server for available contents around the user's coarse location

Sparcl (currently) is only for querying content, and it cannot add new contents to SCD!

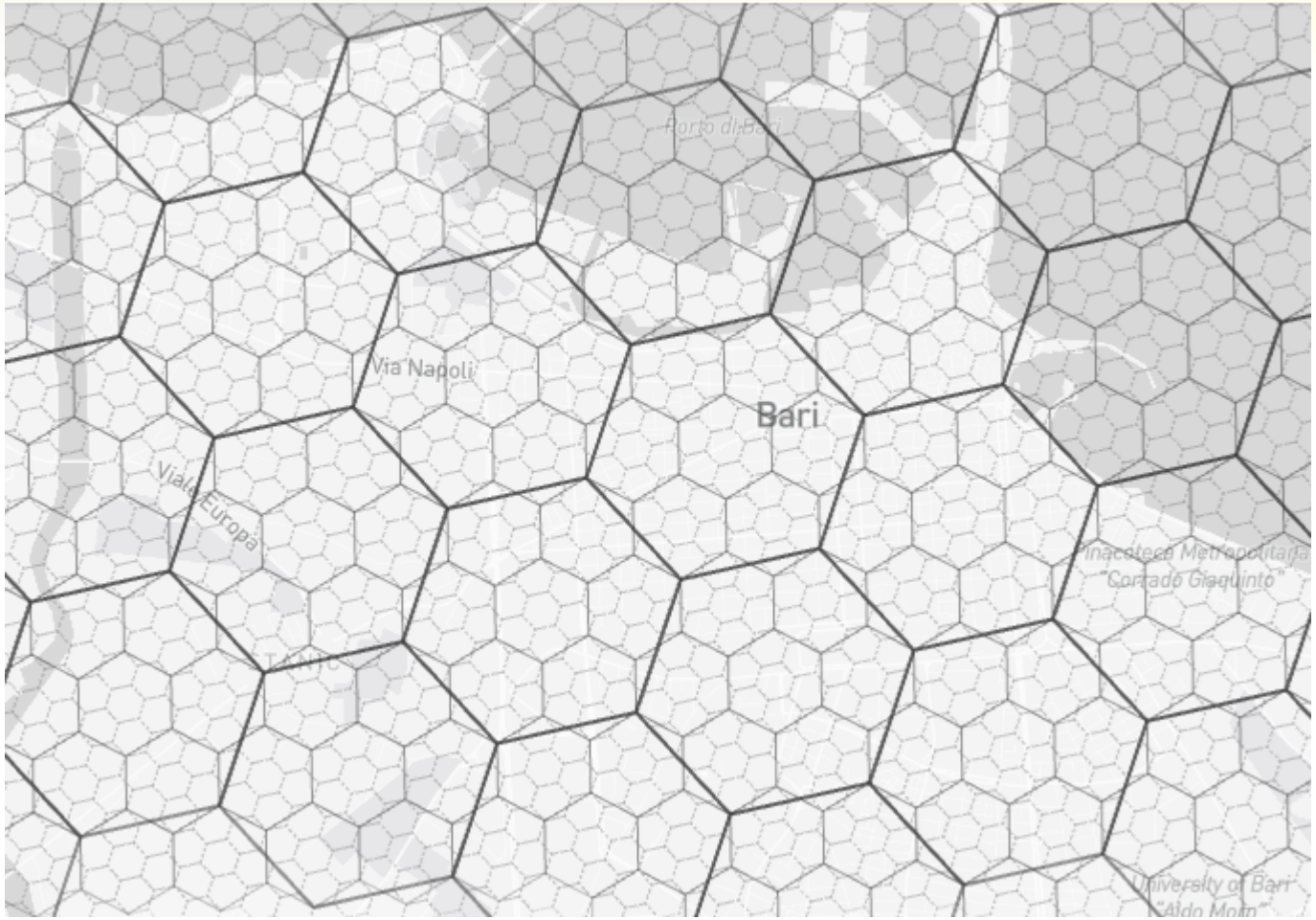
We also provide a service/content administration GUI at https://oscp-admin.orbit-lab.org

H3 index

Hierarchical location index developed by Uber
Multilevel hexagonal grid, down to ~1 m^2 resolution
Easy search in all directions
Neighbor cells have the same distance

Allows search without exposing the user's GPS location

OSCP uses H3 indices in all queries
(next version will support general GeoJSON instead)



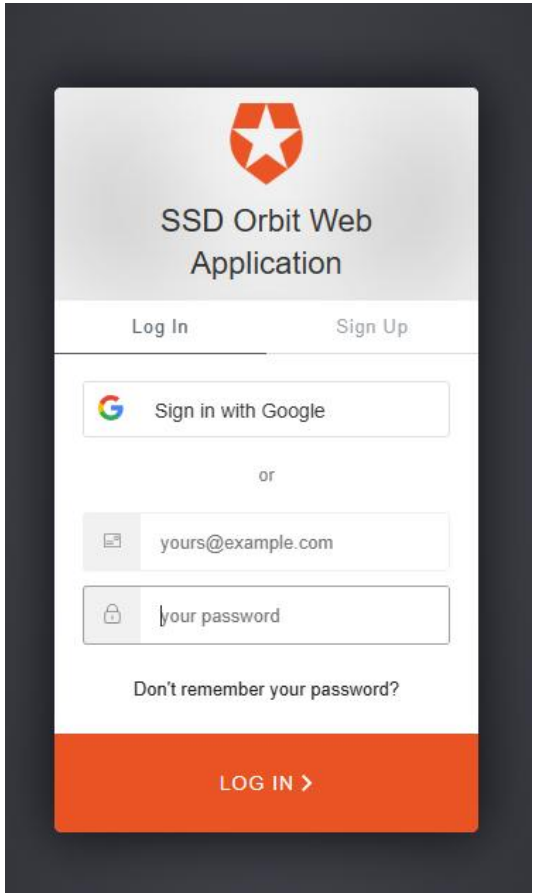
https://h3geo.org/

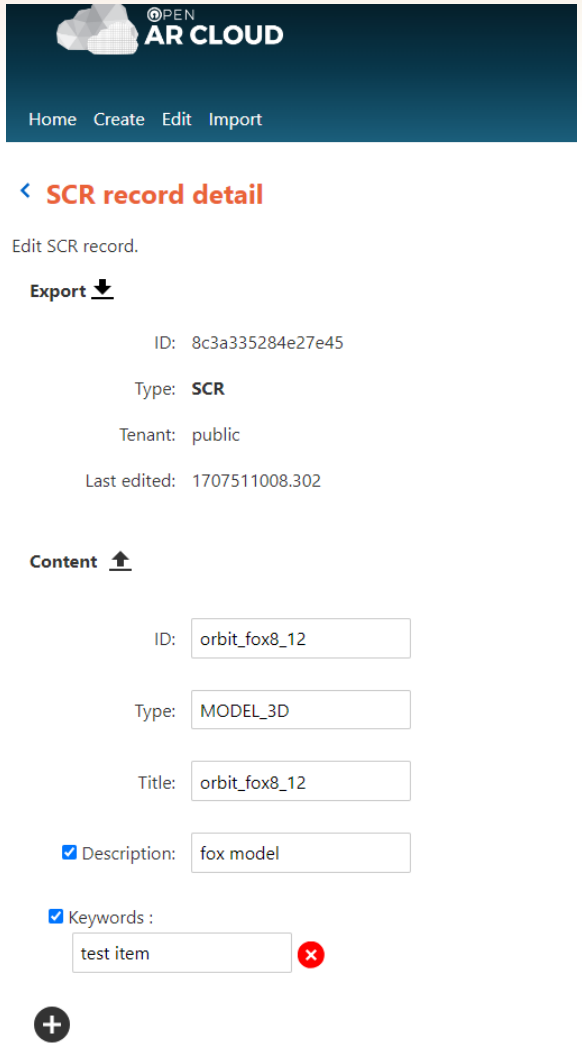
ISMAR2026 location H3 index (level 8): `881e880d5dfffff`

oscp-admin

https://oscp-admin.orbit-lab.org

Try it!





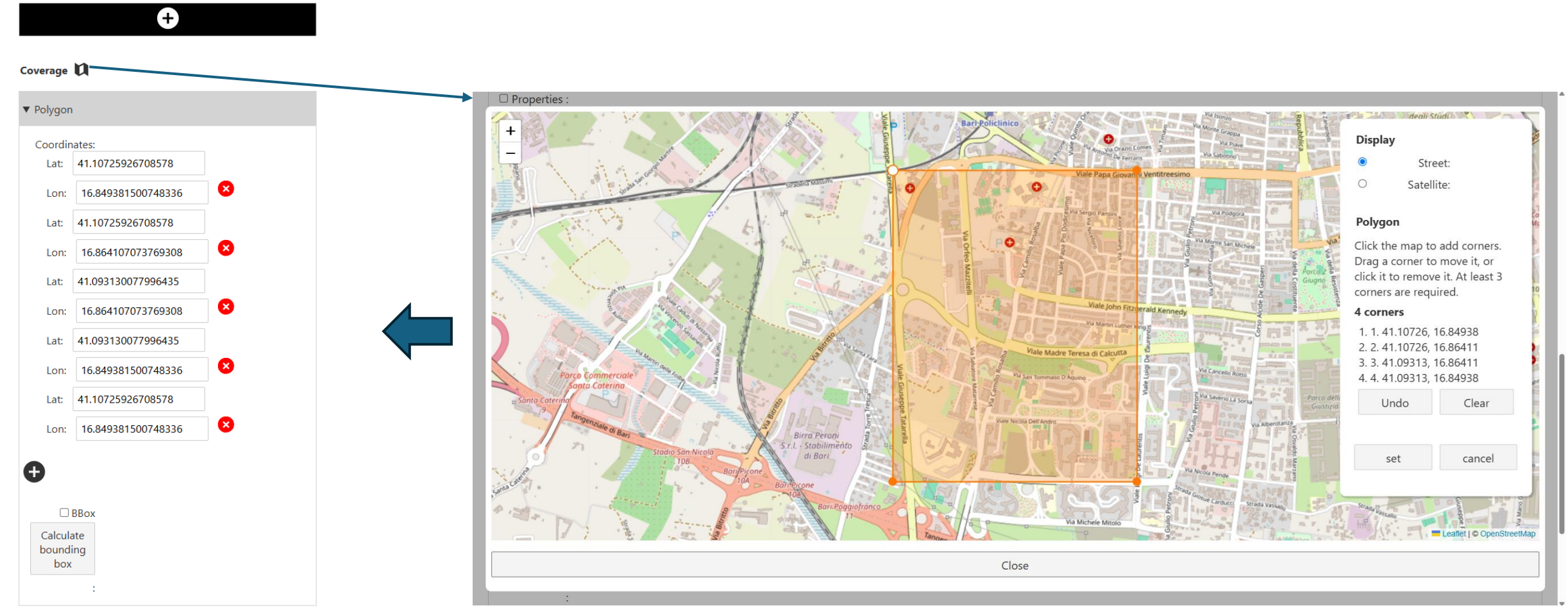
Use Google for login. The app reads your name and email address. All records are public!

Test services at ISMAR2026

https://oscp-admin.orbit-lab.org/ssd

Coverage area as GeoJSON, Query area as H3 index

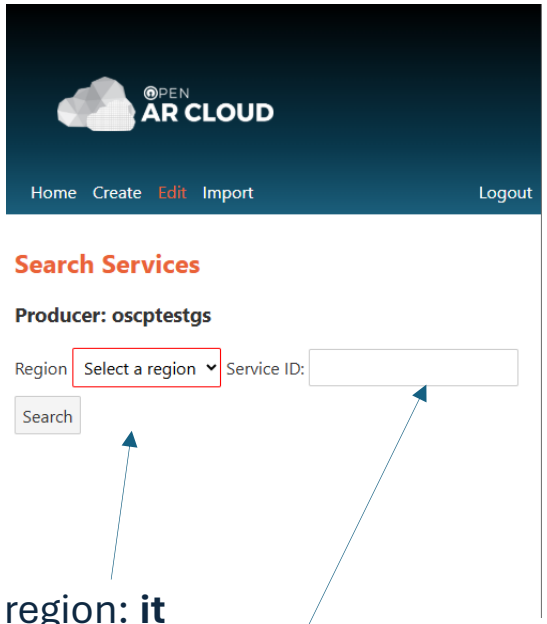
H3 index: 881e880d5dfffff



Test services at ISMAR2026

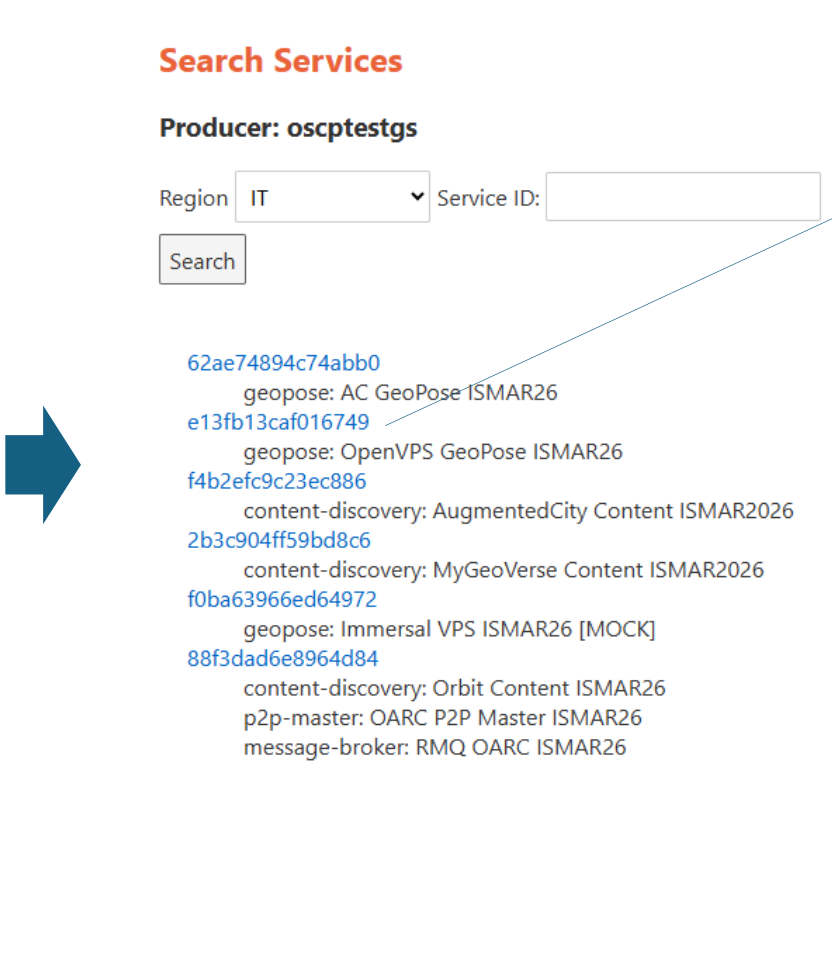
https://oscp-admin.orbit-lab.org/ssd

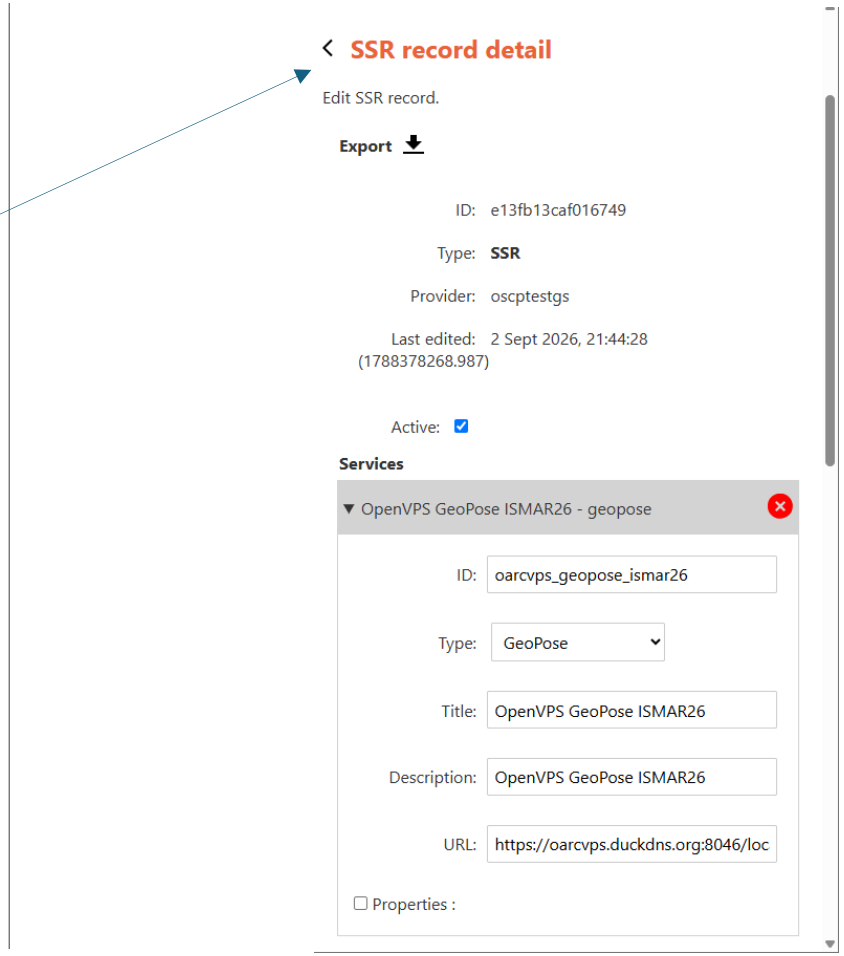
Create SSR



region: **it**

empty and click search





Test services at ISMAR2026

Search and validate

Now we can try to query these services:

a) by clicking Edit in `https://oscp-admin.orbit-lab.org/ssd`
setting **it** as region, and clicking search

b) via Swagger GUI: `https://ssd.orbit-lab.org/swagger/`
setting **it** as region and `881e880d5dfffff` as H3 index

c) directly from SSD: https://ssd.orbit-lab.org/it/ssrs?h3Index=881e880d5dfffff
note that client apps assemble this URL and use SSD this way

Test content at ISMAR2026

Create SCR

Create Spatial Content Record

Enter data for new SCR record.

Export

Type: **SCR**

Topic: history

only **history** (for Database)

SCR record detail

Edit SCR record.

Export

ID: bf950220e3c9cfe7

Type: **SCR**

Tenant: public

Last edited: 4 Oct 2026, 10:42:55 (1791103375.97)

your ID for this object

Content

ID: bari_dragon

Type: MODEL_3D

Title: bari_dragon

Description: A dragon at bari

Keywords :

Placekey:

only **MODEL_3D** (for Sparcl)

https://oscp-admin.orbit-lab.org/scd

GeoPose

Position:

Latitude: 41.09890288216545

Longitude: 16.85477321654602

Height: 10

Quaternion:

X: 0

Y: 0

Z: 0.9238795

W: -0.382683

Euler angles (degrees):

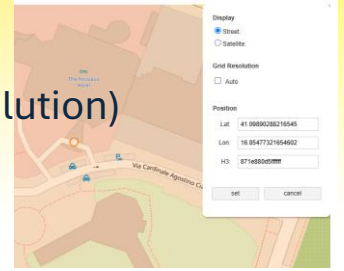
East: 0

North: 0

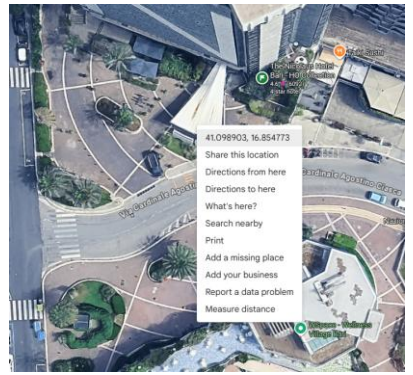
Up: -135

Limitation: height is not true altitude yet

built-in map (limited resolution)



more precise geolocation from GMaps



Orientation (ENU)

Refs :

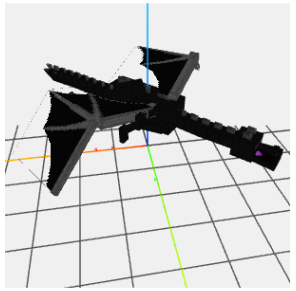
Content Type: model/gltf+json

model/gltf+json

URL: https://raw.githubusercor

URL: must be directly accessible

https://raw.githubusercontent.com/balazs-ladjanszki/3d/main/dragon_scaled.glb



custom key-value pairs

Definitions :

scale %

Size:

Bounding Box:

Save

Test content at GMU

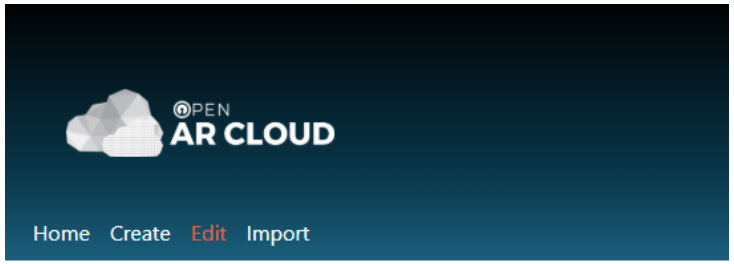
Search and validate

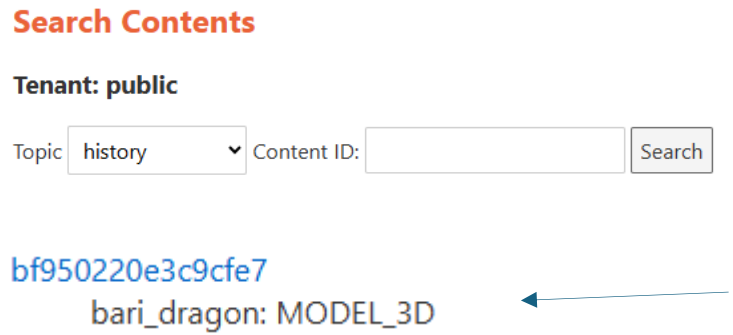
Direct query: https://scd.orbit-lab.org/scrs/history?h3Index=881e880d5dfffff
note that client apps assemble this URL and use SCD this way



history
for now

empty,
press search



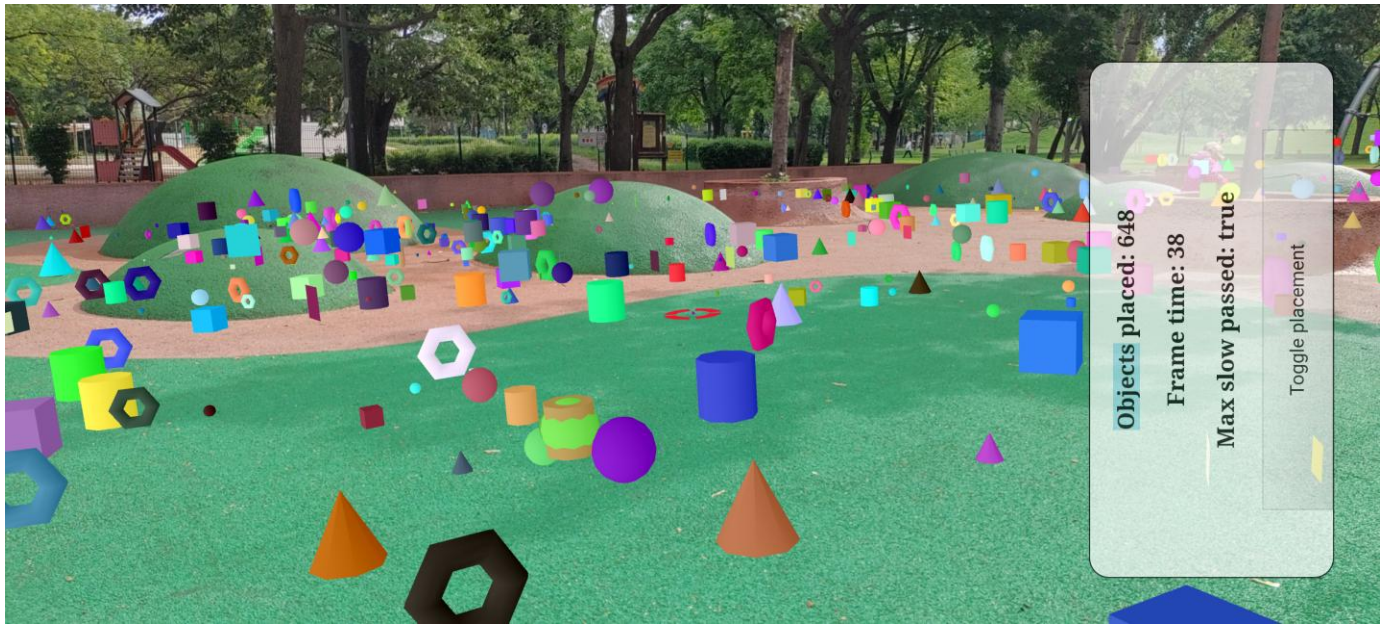


our new object

OGL content

Currently supported contents:

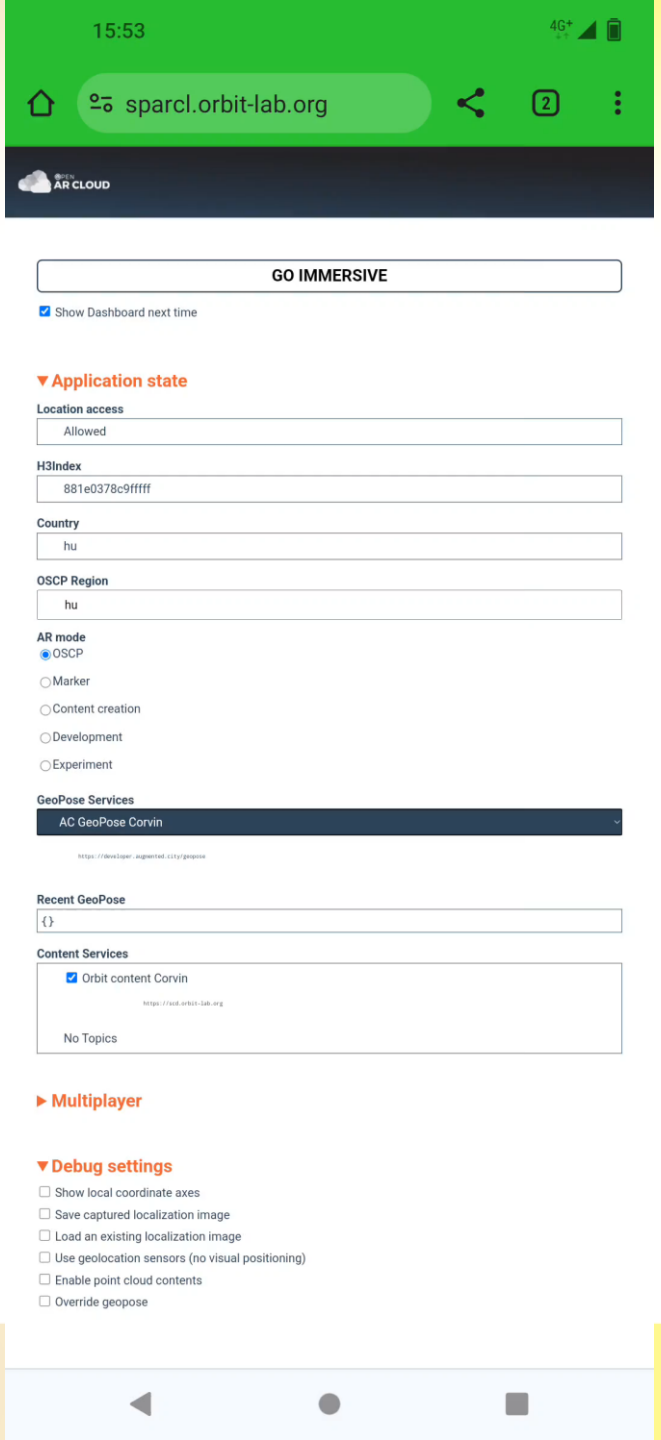
- placeholders
- primitives from textual descriptions
- GLTF/GLB models (texture OK, but no complex shaders)
- iframe content loaded via URL



`https://github.com/oframe/ogl`

OGC Point of Interest


```
{
   "type": "Feature",
   "geometry": {"type": "Point",
   "coordinates": {48.22459095, 16.35021388}},
   "properties": {
      "featureID": 693842,
      "name": {"name" : "Volksoper Wien"},
      "hasFeatureOfInterest": {"href": "https://volksoper.at"},
      "hasPayload": {
        "usesSchema" : "egpoi.json",
        "hasDefinition" : "egpoi",
        "isPublic:" : true
      },
      "openingHours": {
          "openingHoursLines": "Tu-Fr 9:00-17:00",
          "openingHoursFormat": "schemadotorg"
      },
      "category": {
          "category": "theatre",
          "categorySystem": "osm"
      }
   }
}
```

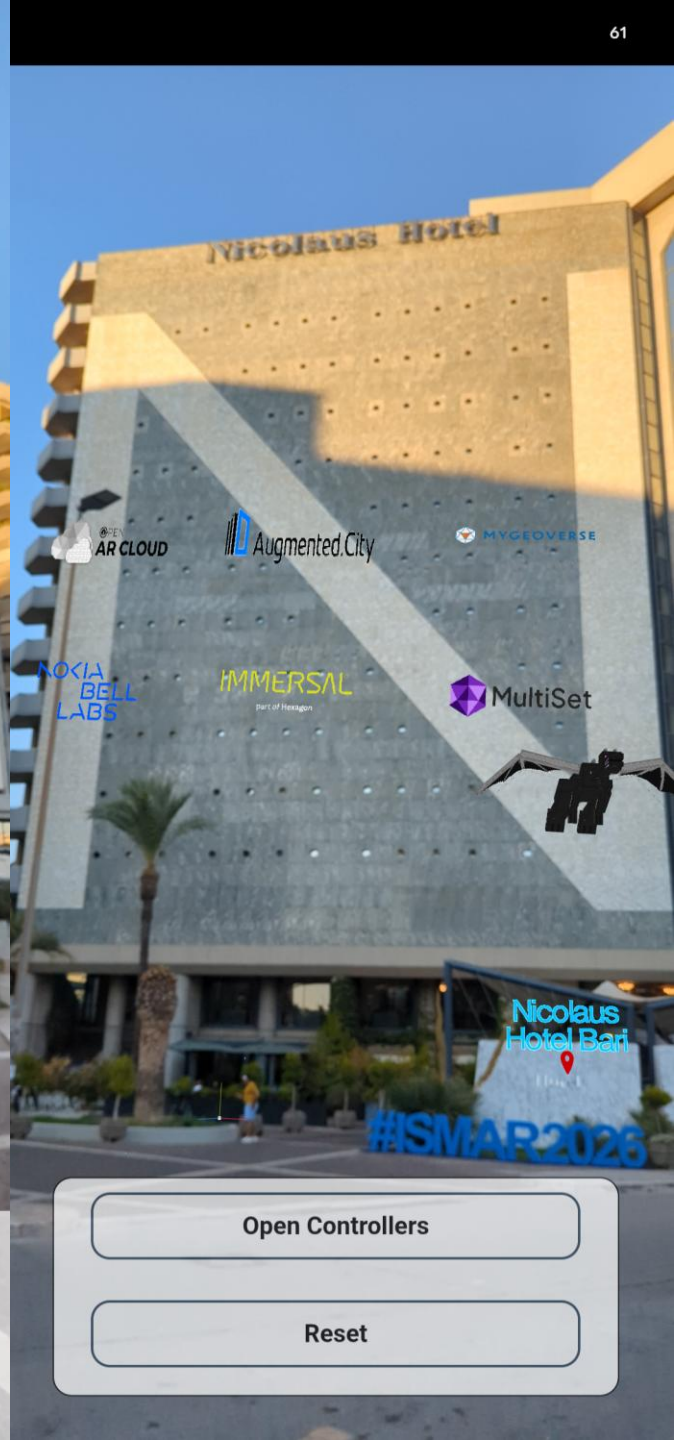
Demo at the venue

https://sparcl.orbit-lab.org

Try it!







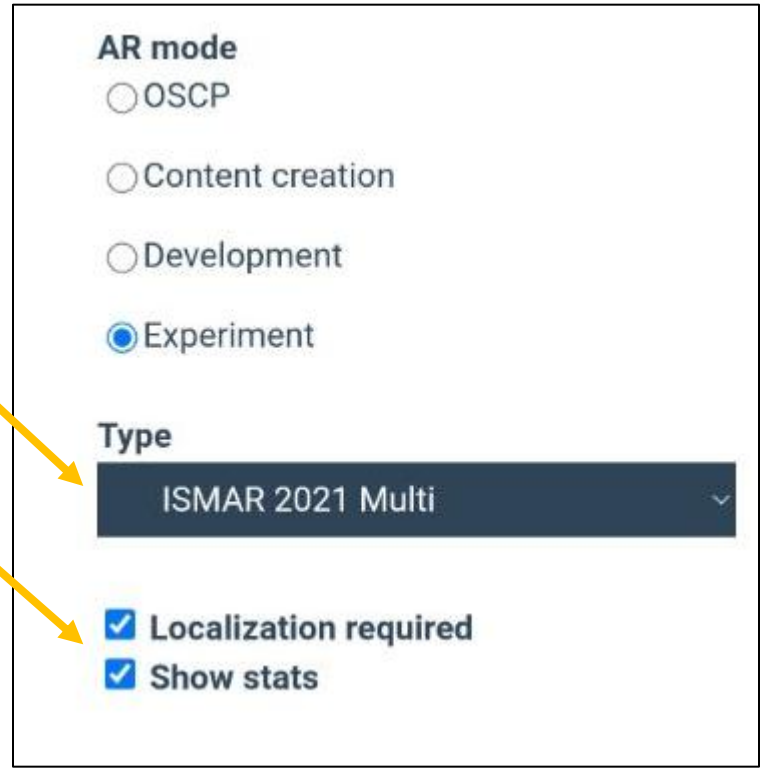
Extending sparcl

Experiment mode:

- define an experiment name and add to `Selector.svelte`
- `Settings.svelte`: this will be loaded into the main Dashboard
- `Viewer.svelte`: extend standard functionality

All our demos are implemented as experiments

(see `ismar2021` branch)



`https://github.com/OpenArCloud/sparcl`

`https://openarcloud.github.io/sparcl/architecture/experiments.html`

context-aware animated content (ThreeJS)



Video: https://youtu.be/23kC43Xg2q4

OPEN AR CLOUD

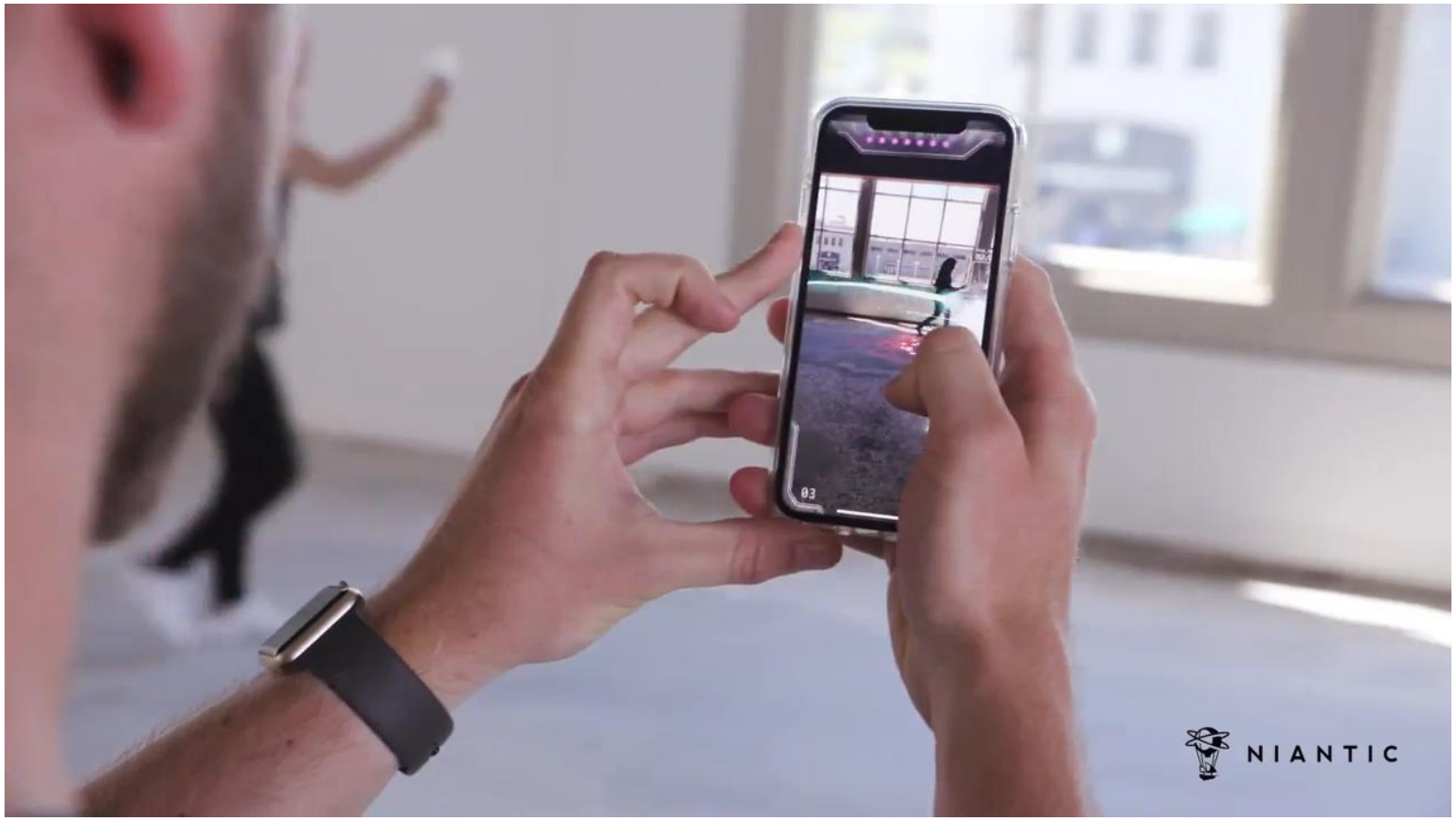
5 Oct 2026 Bari, Italy

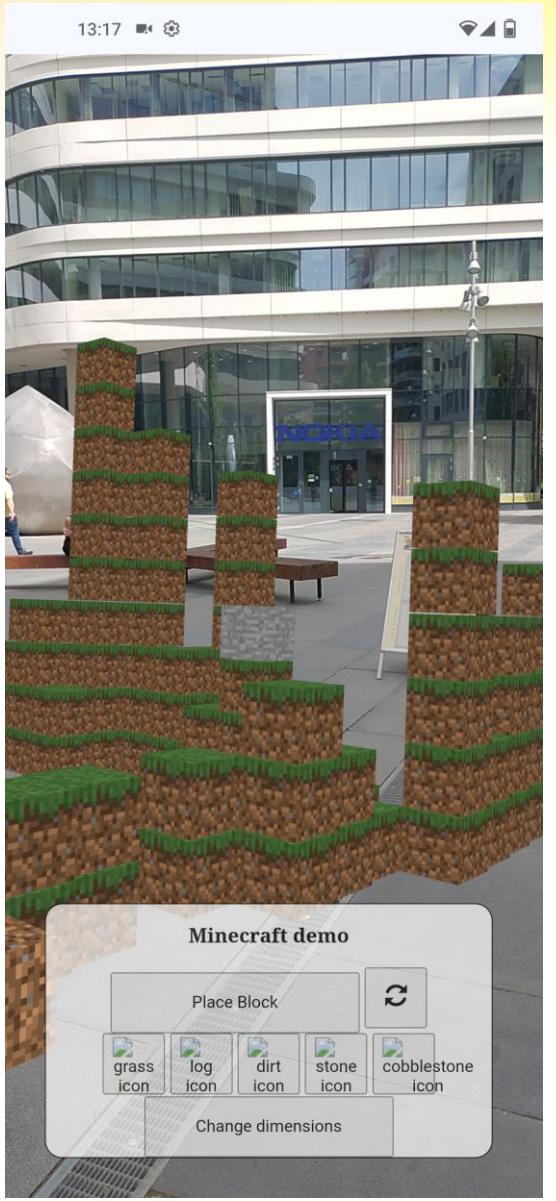
spARcl advanced features

Dr. Gábor Sörös

Nokia Bell Labs & Open AR Cloud

Multiplayer games in augmented reality





Peer-to-peer multiuser experiences

all clients run the same code

multiple transmission options (PeerJS, RabbitMQ, etc.)

local network parameters (e.g., rendezvous server) are retrieved from SSD

events are shared and logged: player poses, objects created, etc.

local-first design (~ collaborative document editing) using Automerge

headless client (lives forever) stores history

- offline periods are OK
- CRDT: conflict-free replicated datatypes
- automatic merging of diverged histories
- every new client can get the full history

https://github.com/OpenArCloud/sparcl/blob/main/docs/guides/multiuser.md

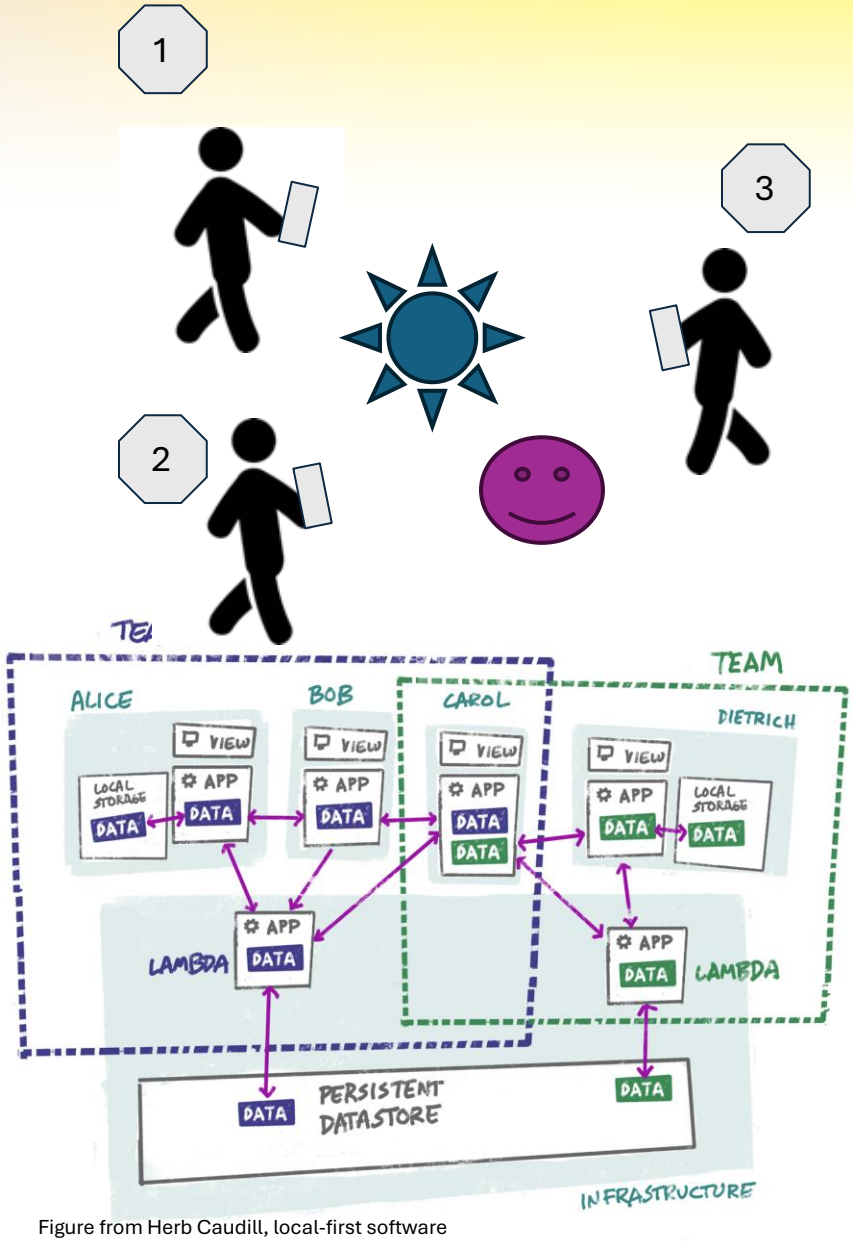
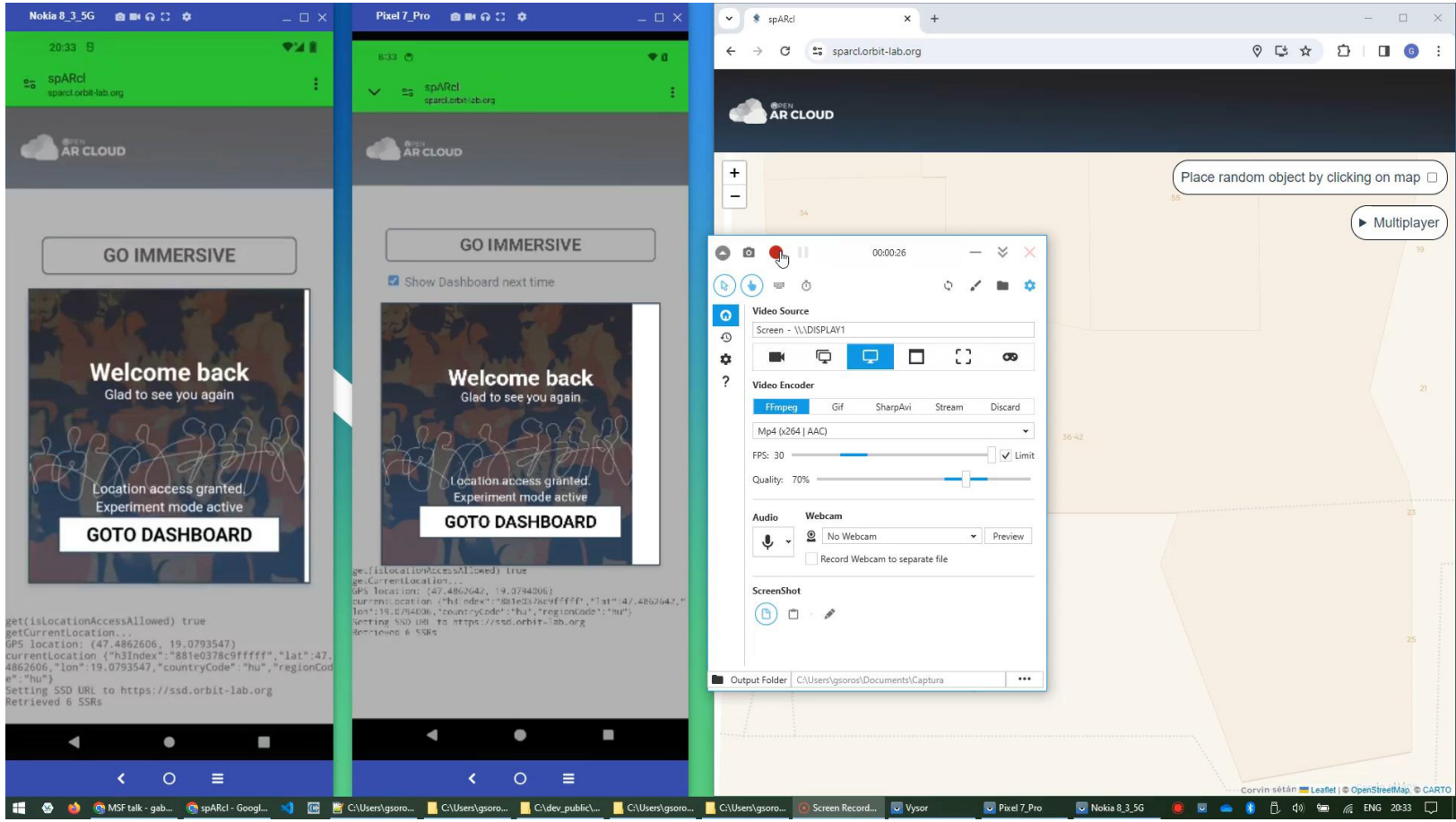


Figure from Herb Caudill, local-first software

Nokia-OARC peer-to-peer AR object sharing

- All devices run own motion tracking, co-localize via VPS into shared geo-coordinate system (GeoPoseProtocol)
- No proprietary AR anchors
  but only GeoPose
- WebXR gives us hit test target point on the ground (local pose). We calculate the GeoPose of the hit point
- We generate a random object descriptor (shape, size, color) as JSON. We fill it into a spatial content record (SCR) with the GeoPose and share it
- We draw the received object by the renderer
- Works in both directions



Video: https://youtu.be/JTqQ0HALyXY?si=Y46ovfkEWIjlVFL0&t=173

Collaborative block-world builder

Code: https://github.com/balazs-ladjanszki/OARC_sparcl/tree/minecraft2

App: https://oarcvps.duckdns.org:8044/

Try it!

Blocks shared over P2P

Players need to start in the same H3 cell

Contents are persisted until one player is online

To restart building, everyone must clear the automerge history





Videos:
https://youtube.com/shorts/RgWFm2iHCRc
https://youtu.be/ixnEXkaeD6c

B.Sc project of Balázs Ladjánszki, TU Budapest, 2024

P2P service and the headless client

We need to register a service of type **`p2p-master`** in the SSD for the coverage area. The clients will receive all necessary information (e.g., signaling server URL and peer id of the headless client to connect to) from the spatial service record.

To keep the history of the experience at a location, a long-running, headless client can be started which merely serves as a bookkeeper. The code for this is already withing sparcl, it is seen as an equal peer in the P2P network but it has no AR features and no UI at all.

Start a headless client:
`https://sparcl.orbit-lab.org?headless=true`
(simply specify `peerid` as a parameter)

Start a headless client and use specific signaling server and port:
`https://sparcl.orbit-lab.org?headless=true&signal=` YOUR_URL `&port=` YOUR_PORT
(besides `peerid`, also specify `signal` and `port`)

GeoPose streams

Try it!

Video: https://youtu.be/Z09ilsnFpSw

Register **Message-Broker** service (RabbitMQ)

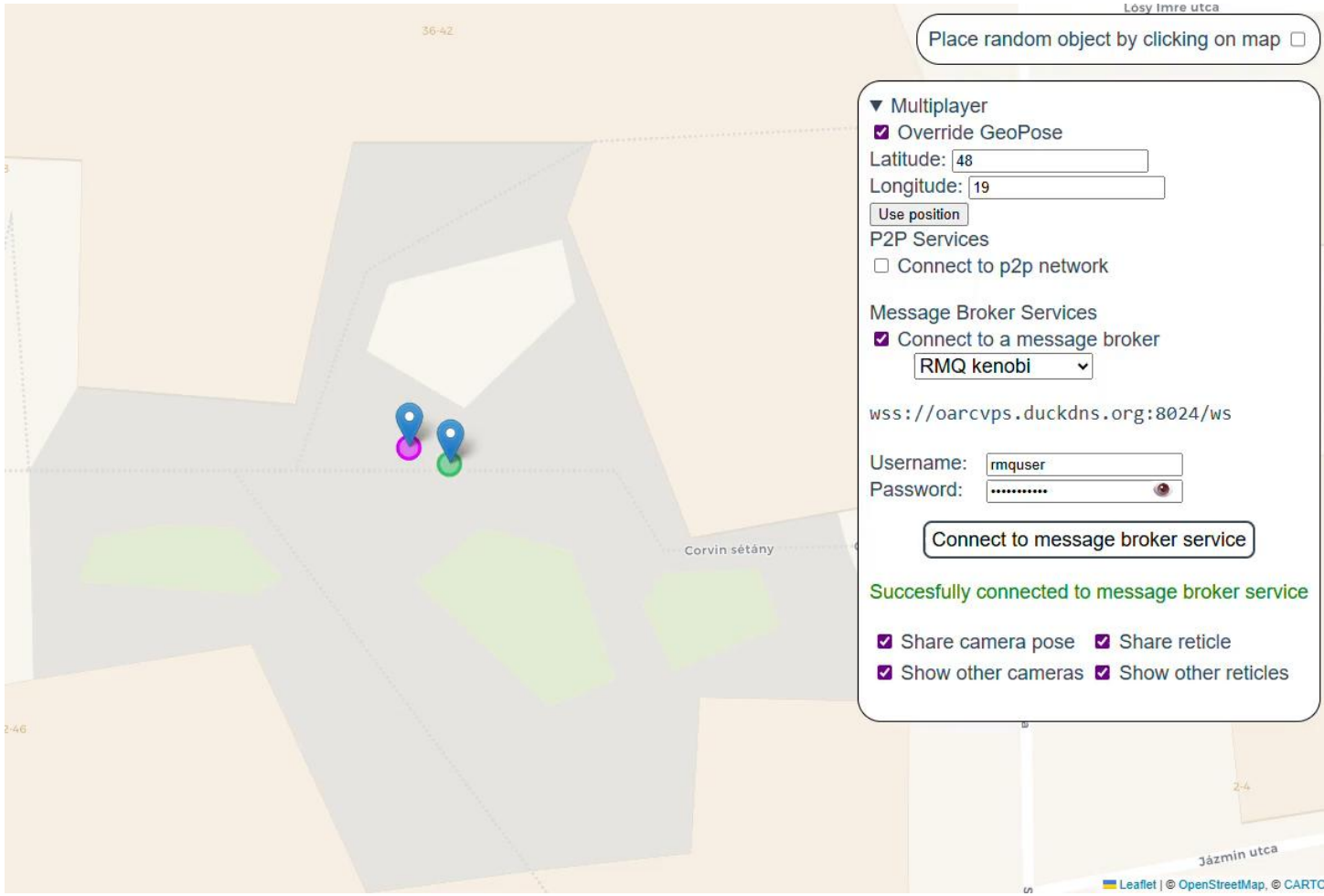
Password protection:
add authentication: password in SSR
(ask me for password)

If service available, spARcl shows RMQ widget

Multiplayer username can be set or extracted from the login email

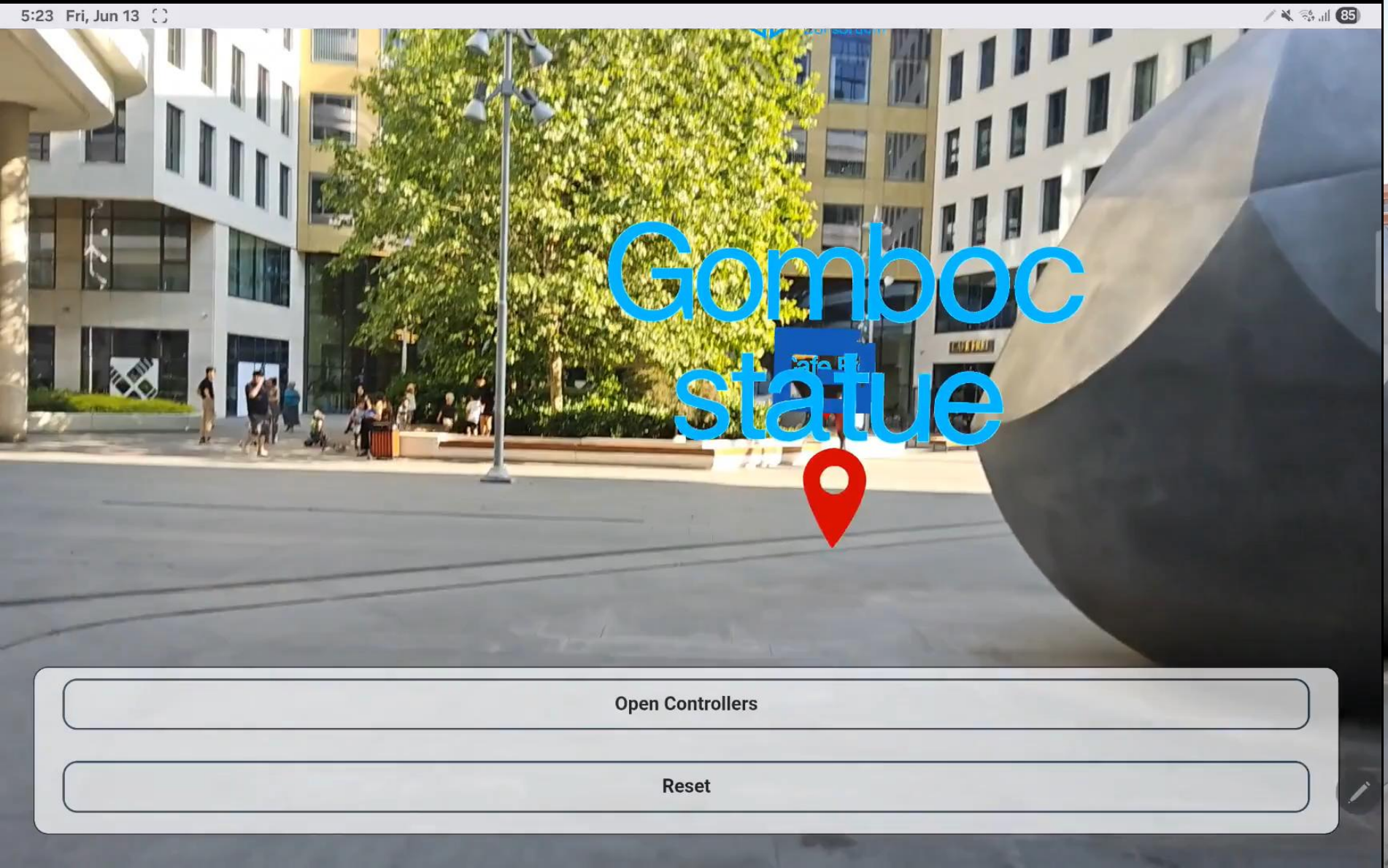
RabbitMQ (AMQP) message broker connection

- Streaming camera GeoPose to others
- Subscribing to GeoPose stream of others, visualizing them in AR (floating name tag, avatar color, etc)
- Other streams: IoT sensor data, robot pose, robot waypoints, etc.



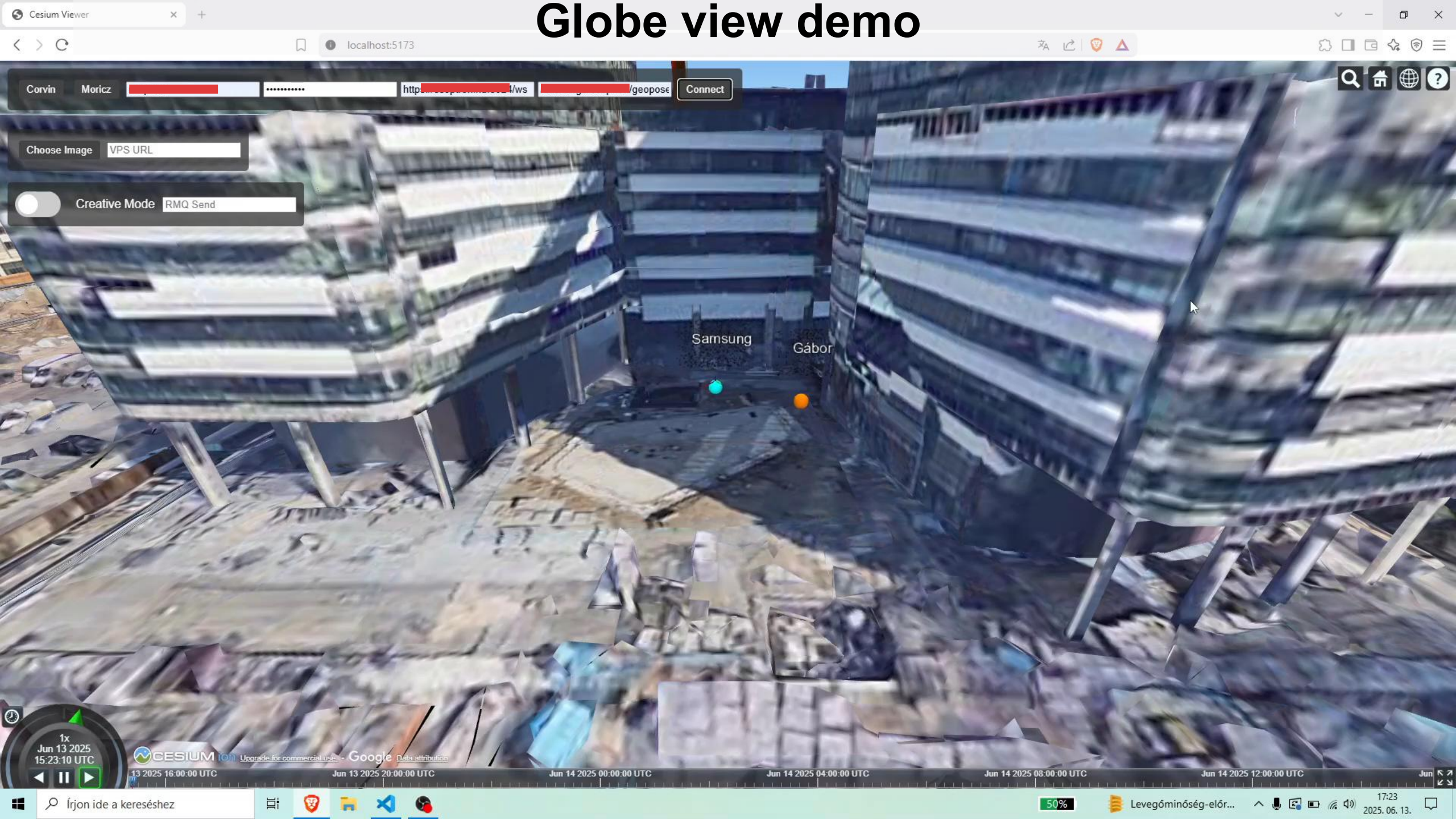
Multiplayer demo

https://youtu.be/0Sjo_IBs1K8
https://youtube.com/shorts/IJRx_ehYzHI





Globe view demo



Memento Experiment

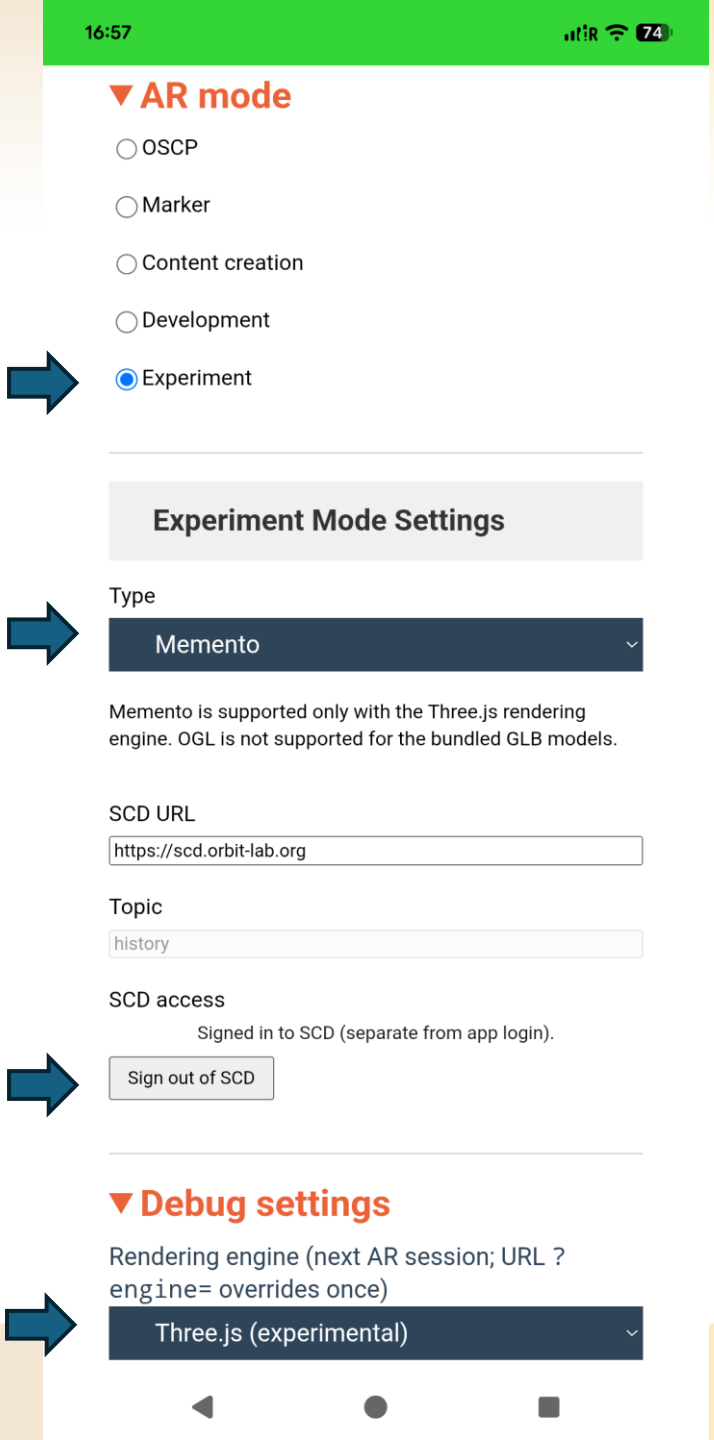
Leave behind a smiley with your name

Choose from 100 built-in models

Write access to SCD

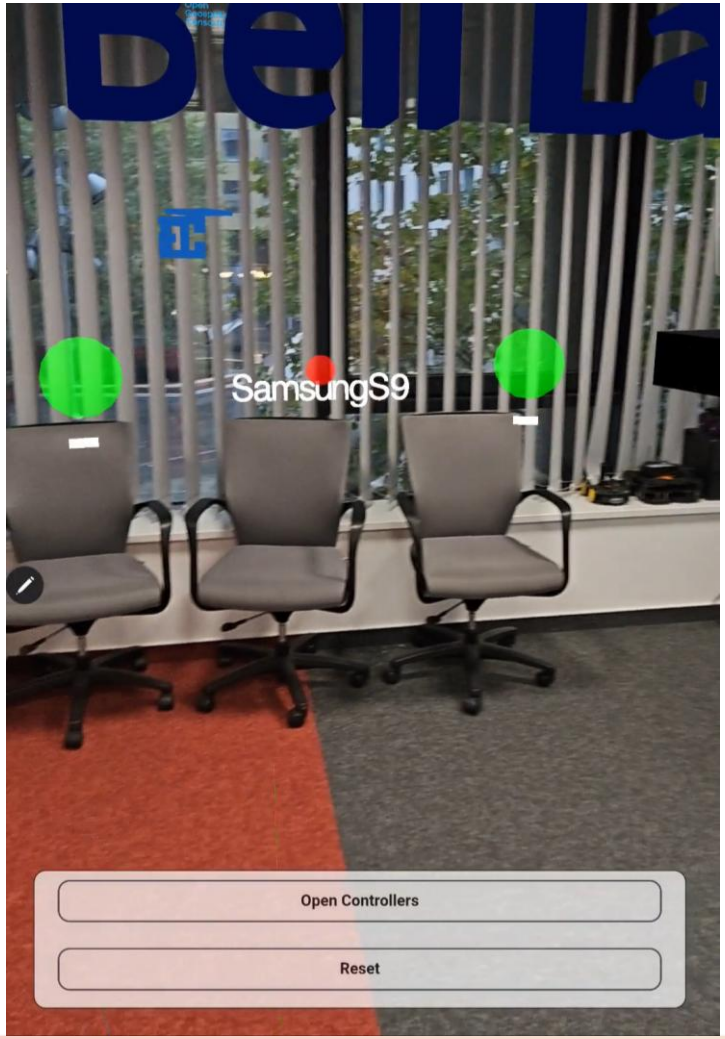
(requires Auth0 login)

Only with ThreeJS engine (OGL does not support instanced GLTF)

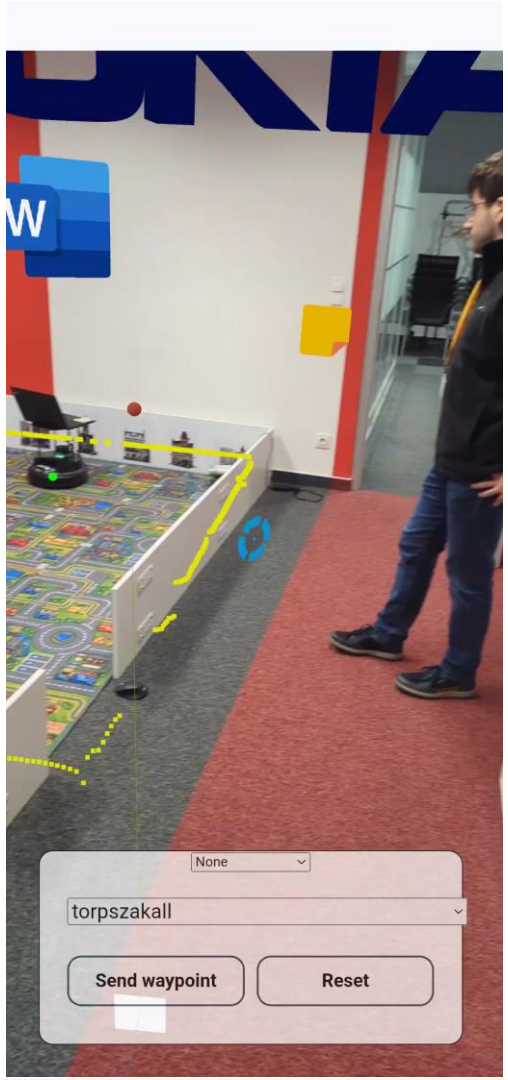




Nokia Budapest office AR examples



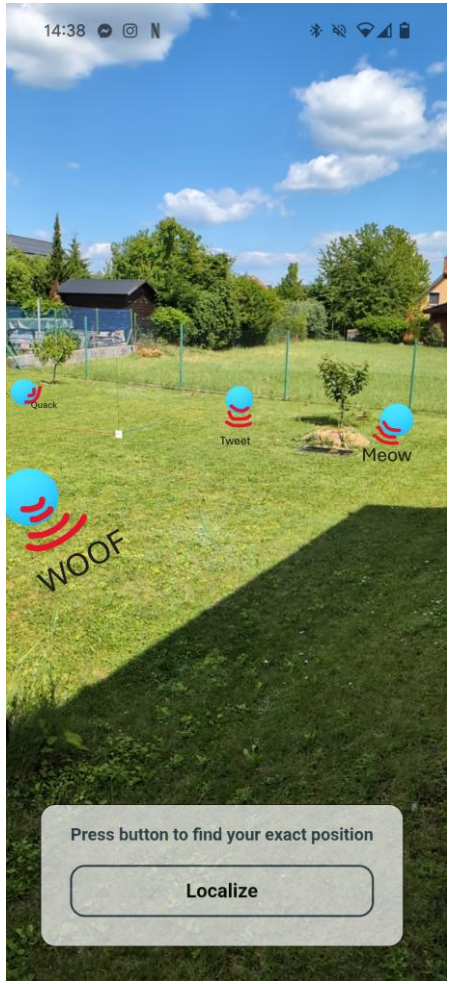
Indoor visual localization
Seat reservation
Indoor navigation
Room reservation
Other digital assets

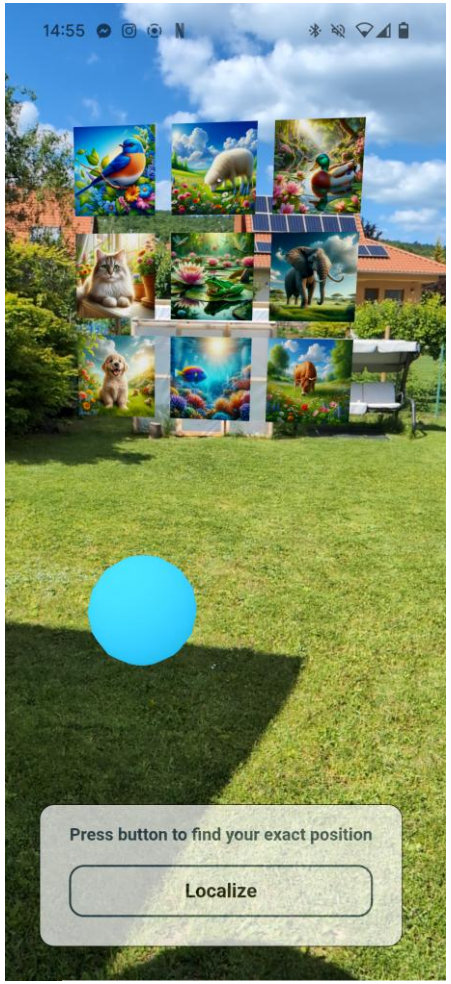


Human-robot co-localization
IoT sensor streaming
Robot pose streaming
Lidar sensor streaming
Robot control via waypoints in AR

Sound search game

Which animal is in which bush?









B.Sc. project of Imola Szin, TU Budapest, 2025

Contributors:



Michael Vogt
(freelancer)



Gábor Sörös
(research scientist)



Tamás Rigó
(research engineer)



Asmaa Thabet
(engineering intern)



Andor Kovács
(engineering intern)

Q & A

Test it at
https://sparcl.orbit-lab.org
spARcl docs: https://openarcloud.github.io/sparcl/
spARcl code: https://github.com/OpenArCloud/sparcl

References

Gábor Sörös, Andor Kovács, Dénes Bisztray, Asmaa Thabet, Balázs Ladjánszki, James Jackson, Christine Perey, Alina Kadlubsky - Demo: Open Visual Positioning and Discovery Services for Location-based Augmented Reality, In Companion of the the 2025 ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp Companion '25), Helsinki, Finland, Oct 2025

Gábor Sörös, James Jackson, Michael Vogt, Mikel Salazar, Alina Kadlubsky, Jan-Erik Vinje - An Open Spatial Computing Platform, In Proc. 2024 IEEE International Conference on Metaverse Computing, Networking, and Applications (MetaCom), Hong Kong, Aug 2024

Gábor Sörös, Sebastian Eger - Poster: Mobile Robots and Sensor Streams in an Open Spatial Computing Platform, In 2024 EuCNC & 6G Summit, Antwerp, Belgium, June 2024

Gábor Sörös, John Nilsson, Nan Wu, Jennifer Shane, Alina Kadlubsky - Demo: End-to-end open-source location-based augmented reality in 5G, In IEEE Symposium on Mixed and Augmented Reality (ISMAR), Singapore, Oct 2022

James Jackson, Michael Vogt, Gábor Sörös, Mikel Salazar, Sergey Fedorenko - Demo: The first open AR cloud testbed, In IEEE Symposium on Mixed and Augmented Reality (ISMAR), Bari, Italy, Oct 2021

Links

GeoPose 1.0 Website

OGC draft POI Conceptual Model

OARC Demonstration of PoI, VPS and GeoPose

The First Open AR Cloud Testbed (ISMAR 2021)
https://www.youtube.com/watch?v=E-kSY5JaIVA

End-to-End Open-Source Location-based Augmented Reality on 5G (ISMAR 2022)
https://www.youtube.com/watch?v=8BPHI5WMmYI

A WebXR Client for an Open AR Cloud (MSF tutorial) https://youtu.be/wok0clWDtfo

OARC PoI interoperabiltiy demo 2024 https://medium.com/openarcloud/historical-interoperability-demo-for-poi-data-on-oscp-at-ogcs-128th-member-meeting-in-delft-9ca063299d36

Martin Kleppmann - Local-first software: You own your data, in spite of the cloud,
https://www.inkandswitch.com/local-first/

OPEN AR CLOUD

5 Oct 2026 Bari, Italy

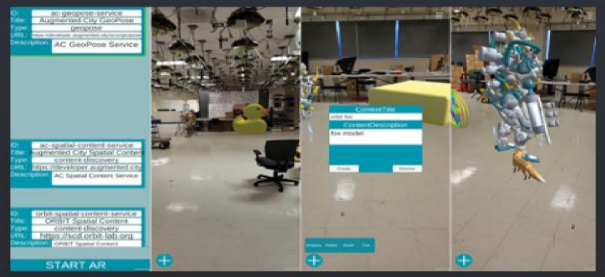
OARC Unity client

End-to-end open-source location-based augmented reality in 5G

Gábor Sörös, John Nilsson, Nan Wu, Jennifer Shane, Alina Kadlubsky

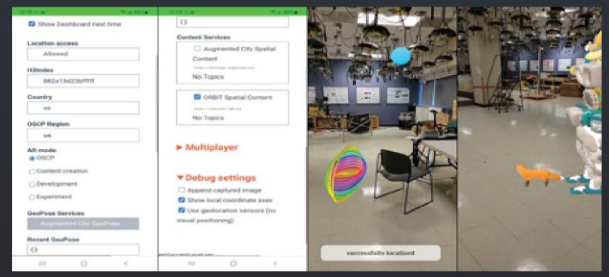
We present a fully open source, end-to-end solution for placing, storing, and re-discovering geo-anchored AR contents in the real world. We advanced components of the Open Spatial Computing Platform (OSCP) and deployed it in a 5G testbed.
We created a client in Unity and built representative demo applications that allow virtual objects to persist throughout space and time. We created a data collector application for spatial mapping, and we connected the OSCP with an open-source visual positioning system. Furthermore, we created tutorials on the setup to foster its adoption in the scientific community as a platform for location-based AR experiments.

Experiment 1



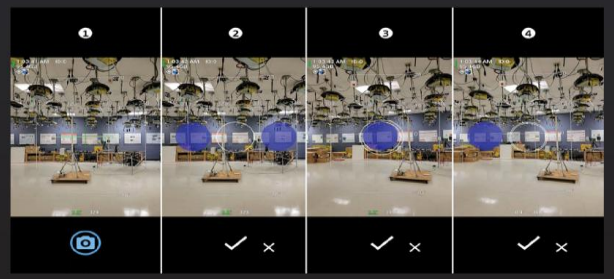
Open-source Unity client (Aurora viewer) localizing in the WINLAB, discovering services, discovering the existing contents, and creating new content (Fox). All objects are linked to GPS coordinates via OGC GeoPose.

Experiment 2



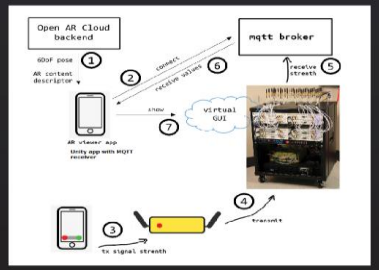
Open-source WebXR client (Sparcl) localizing in the WINLAB, discovering the available services, discovering the existing contents, and discovering the newly created content.

Image capture app for VPS mapping



Open-source camera app showing the compass direction, level lines, and hints about the next best view to capture. When the user rotates the device to align over the next blue spot, the next image is automatically captured. The images (along with GPS coordinates and other metadata) are stored in the universal Kapture format.

Experiment 3



Demonstrating augmented reality visualization of real-time IoT sensor data. We retrieve sensor values, and show them in dynamically generated UI widgets above the physical object. For example the signal strength of a remote-controlled radio.

https://github.com/OpenArCloud

Thank you !

Building Interoperable Location-Based Augmented Reality with the Open AR Cloud



Gábor Sörös

gabor.soros@openarcloud.org